

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



XỬ LÝ NGÔN NGỮ TỰ NHIÊN
CS221.F31.LT.TTNT

**KHẢO SÁT VÀ ĐÁNH GIÁ HIỆU NĂNG CÁC MÔ HÌNH
TRANSFORMER TIỀN HUẤN LUYỆN TRONG BÀI TOÁN
ĐỌC HIỂU VĂN BẢN VÀ TRẢ LỜI CÂU HỎI TIẾNG VIỆT**

*Monolingual vs. multilingual Transformers for Vietnamese
extractive MRC: a diagnostic evaluation on UIT-ViQuAD 2.0*

BÁO CÁO ĐỒ ÁN MÔN HỌC

GVHD: TS. Đặng Văn Thìn

Nhóm thực hiện: Nguyễn Quang Lâm — 25210289
Trần Trọng Tấn — 25210334
Lê Quang Thi — 25210337
Võ Cẩm Thu — 25210342

TP. Hồ Chí Minh, tháng 9 năm 2026

Lời cảm ơn

Nhóm xin cảm ơn TS. Đặng Văn Thèn, giảng viên môn *Xử lý ngôn ngữ tự nhiên* (CS221), đã định hướng đề tài và nhấn mạnh xuyên suốt môn học rằng phương pháp phải được chọn dựa trên đặc điểm của dữ liệu, và rằng phân tích lỗi là phần quan trọng nhất của một nghiên cứu NLP — hai ý đã định hình cách nhóm đặt câu hỏi và trình bày đồ án này.

Nhóm cũng cảm ơn Phòng thí nghiệm NLP@UIT đã công bố bộ dữ liệu UIT-ViQuAD 2.0, VinAI Research đã công bố PhoBERT, và cộng đồng mã nguồn mở HuggingFace, PyTorch, scikit-learn và `pyvi` đã cung cấp hạ tầng để một nhóm sinh viên có thể huấn luyện và đánh giá mô hình Transformer trên máy tính cá nhân.

TP. Hồ Chí Minh, tháng 9 năm 2026

Nhóm thực hiện

Tóm tắt

Đề án khảo sát hai mô hình Transformer tiền huấn luyện cho bài toán **đọc hiểu trích xuất tiếng Việt** trên UIT-ViQuAD 2.0: **PhoBERT** — đơn ngữ, nhận đầu vào *đã tách từ* — và **XLM-R** — đa ngữ, dùng subword trên văn bản thô. Hai mô hình khác nhau đúng ở khâu tách từ, nên câu hỏi trung tâm là: đặc điểm tách từ của tiếng Việt ảnh hưởng thế nào đến việc chọn giữa hai họ mô hình?

Hai mô hình được fine-tune trong cùng điều kiện; epoch được chọn trên một tập dev tách từ train, và validation (3.814 câu) được chấm đúng một lần. Để đưa PhoBERT — vốn không có offset ký tự — vào bài toán trích xuất, nhóm cài một lớp tokenizer dựng offset từ bước tách từ `pyvi`. Ngoài EM/F1, nhóm đo: trần EM do biên từ (không cần mô hình), phân loại lỗi tự động, precision/recall của hành vi từ chối, và kiểm định cặp McNemar/bootstrap.

Kết quả. Điểm tổng của hai mô hình không khác nhau có ý nghĩa (EM 51,36 so với 52,49; chênh lệch -1,13, CI 95% [-2,88; 0,60]), nhưng che hai hồ sơ ngược nhau: PhoBERT hơn 14,89 điểm EM trên câu có đáp án, còn XLM-R hơn -37,73 điểm trên câu không có đáp án. Giả thuyết ban đầu (PhoBERT yếu vì lỗi tách từ) *sai* ở dạng tổng quát: chỉ 1,24% đáp án cắt ngang một từ, nên hiệu ứng này bị chặn ở 0,75 điểm EM, và phần lớn các trường hợp đó là lỗi của bộ tách từ. Điều quyết định là hành vi từ chối: PhoBERT chỉ từ chối 5,66% số câu, vì loss trên lớp “không có đáp án” của nó tăng khoảng 7 lần sau epoch 1 trong khi loss trên vị trí đáp án vẫn giảm — hiện tượng lặp lại ở hai tốc độ học. Cuối cùng, kiểm định bộ stress-test 250 câu của nhóm cho thấy chỉ 15/120 câu có đáp án chứa đáp án trong ngữ cảnh, và hệ thống luôn từ chối đúng đầu bảng điểm trên bộ này; bộ được giữ làm đối tượng phân tích, không dùng để xếp hạng.

Mọi con số trong báo cáo được sinh tự động từ tệp `results/*.json` kèm commit.

Từ khoá: đọc hiểu máy, hỏi đáp trích xuất, tiếng Việt, tách từ, PhoBERT, XLM-R, UIT-ViQuAD 2.0, câu hỏi không có đáp án, đánh giá chẩn đoán.

Mục lục

Lời cảm ơn	i
Tóm tắt	ii
Danh sách hình vẽ	v
Danh sách bảng	vi
Danh mục từ viết tắt	vi
1 Giới thiệu	1
1.1 Đặt vấn đề	1
1.2 Phát biểu bài toán	1
1.3 Câu hỏi nghiên cứu	1
1.4 Đối chiếu với yêu cầu môn học	3
1.5 Phạm vi	3
1.6 Đóng góp chính	3
1.7 Cấu trúc báo cáo	4
2 Cơ sở lý thuyết	4
2.1 Bài toán đọc hiểu máy	4
2.2 Tách từ tiếng Việt	5
2.3 Truy hồi câu bằng TF-IDF	6
2.4 Kiến trúc Transformer	6
2.5 Mô hình ngôn ngữ tiền huấn luyện	7
2.6 Mô hình hỏi đáp trích xuất trên encoder	8
2.7 Cửa sổ trượt cho văn bản dài	9
2.8 Tối ưu hoá khi fine-tune	9
2.9 Thang đo đánh giá	10
2.10 Đánh giá chẩn đoán	11
2.11 So sánh hai mô hình trên cùng tập chấm	11
3 Phương pháp	12
3.1 Tổng quan	12
3.2 Giao thức dữ liệu	12
3.3 Hai baseline	13
3.4 Đưa PhoBERT vào bài toán trích xuất	13

3.5	Fine-tuning	14
3.6	Phép đo tương thích biên từ	14
3.7	Phân loại lỗi tự động	15
3.8	Hành vi từ chối như một bộ phân loại	16
3.9	So sánh cặp PhoBERT và XLM-R	16
3.10	Kiểm định bộ stress-test	16
3.11	Truy vết số liệu và kiểm thử	16
4	Thực nghiệm và đánh giá	17
4.1	Môi trường	17
4.2	Dữ liệu	17
4.3	Kiểm định bộ stress-test	18
4.4	Trần EM do biên từ	19
4.5	Kết quả tổng	20
4.6	RQ1 — Mô hình word-level có nhạy với ranh giới từ hơn?	21
4.7	RQ2 — Độ dài ngữ cảnh và độ dài đáp án	23
4.8	RQ3 — Nhận ra câu không có đáp án, hay thiên lệch về từ chối?	25
4.9	RQ4 — Transformer có vượt baseline ở mọi nhóm?	26
4.10	Phân tích định tính	27
4.11	Quá trình huấn luyện và chi phí	27
5	Kết luận	29
5.1	Kết luận	29
5.2	Giới hạn	30
5.3	Hướng phát triển	30
	Tài liệu tham khảo	31
A	Hướng dẫn tái lập	33
B	Kiểm định khung mã ban đầu	33
C	Cấu trúc mã nguồn	34
D	Bảng truy vết số liệu	35

Danh sách hình vẽ

1	Kiến trúc hỏi đáp trích xuất: encoder dùng chung, lớp QA sinh điểm bắt đầu/kết thúc cho mỗi token; token [CLS] đại diện cho lựa chọn “không có đáp án”	8
2	Quy trình. Nhánh xanh dương (PhoBERT) và xanh lá (XLM-R) chỉ khác nhau ở bước tạo token và offset; mọi bước sau dùng chung mã	12
3	Thành phần từng nhóm stress-test. Phần cam là câu có đáp án mà đáp án không nằm trong ngữ cảnh — không mô hình trích xuất nào trả lời đúng được	18
4	Kết quả chính. Đường đứt: điểm tổng của hệ thống luôn từ chối	21
5	Cơ cấu lỗi của từng hệ thống	22
6	EM theo độ dài	24
7	Loss huấn luyện và EM/F1 trên dev theo epoch	28

Danh sách bảng

1	Đặc tả đầu vào, đầu ra	2
2	Bốn câu hỏi nghiên cứu và phép đo trả lời từng câu	2
3	Đối chiếu yêu cầu và nội dung môn học với đề án	3
4	Các dạng bài toán đọc hiểu máy theo dạng đầu ra	4
5	Siêu tham số huấn luyện	15
6	Sáu loại lỗi	16
7	Môi trường phần cứng và phần mềm	17
8	Dữ liệu dùng trong đề án	17
9	Kiểm định tự động bộ stress-test (nguồn: <code>results/stress_test_audit.json</code>)	18
10	Phân loại 33 câu lệch biên từ trên validation (nguồn: <code>results/misaligned_labels.json</code>)	20
11	Kết quả trên toàn bộ validation (3.814 câu; nguồn: <code>results/eval_*_validation.json</code>)	20
12	Hai lần chạy PhoBERT trên validation. *: lần chạy chính (F1 dev cao nhất)	21
13	EM/F1 theo tương thích biên từ của đáp án vàng (câu có đáp án)	22
14	Phân loại lỗi trên toàn bộ validation (số câu; định nghĩa ở Bảng 6)	22
15	So sánh cặp PhoBERT (A) và XLM-R (B) theo EM từng câu. Chênh lệch = $EM(A) - EM(B)$, điểm %	22
16	EM / F1 theo độ dài đoạn văn (âm tiết). †: nhóm dưới 30 câu	23
17	EM / F1 theo độ dài đáp án vàng (âm tiết; câu có đáp án)	24
18	EM / F1 theo phạm vi suy luận (heuristic; chỉ câu có đáp án)	24
19	Hành vi từ chối như một bộ phân loại “không có đáp án” (validation)	25

20	EM trên bộ stress-test theo nhóm: <i>toàn bộ</i> / <i>chỉ câu hợp lệ</i> . Chỉ để minh hoạ — xem Mục 4.3	25
21	Loss trên tập fit theo loại nhãn: “không có đáp án” / “vị trí đáp án” (nguồn: <code>results/loss_probe.json</code>)	26
22	Ví dụ định tính trên validation	27
23	Đường cong huấn luyện. EM/F1 đo trên dev (500 câu ngẫu nhiên tách từ train). *: epoch được chọn	28
24	Các lỗi phát hiện trong khung mã ban đầu	34
25	Nguồn gốc các nhóm số liệu trong báo cáo	35

Danh mục từ viết tắt

Viết tắt	Ý nghĩa
MRC	Machine Reading Comprehension — đọc hiểu máy
EM	Exact Match — khớp chính xác
F1	Trung bình điều hoà của precision và recall (ở đây tính trên token âm tiết)
TF-IDF	Term Frequency – Inverse Document Frequency
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly optimized BERT approach
XLM-R	XLM-RoBERTa — mô hình đa ngữ của Conneau và cộng sự
PhoBERT	Mô hình RoBERTa đơn ngữ tiếng Việt của VinAI
BPE	Byte-Pair Encoding
SQuAD	Stanford Question Answering Dataset
UIT-ViQuAD	UIT Vietnamese Question Answering Dataset
MPS	Metal Performance Shaders — backend GPU của PyTorch trên Apple Silicon
CI	Confidence Interval — khoảng tin cậy

1 Giới thiệu

1.1 Đặt vấn đề

Đọc hiểu máy trích xuất (extractive Machine Reading Comprehension — MRC) là bài toán: cho một đoạn văn và một câu hỏi, chỉ ra đoạn chữ *trong chính văn bản đó* trả lời câu hỏi, hoặc kết luận rằng văn bản không chứa câu trả lời. Đây là thành phần “bộ đọc” của các hệ thống hỏi đáp trên tài liệu và của các pipeline truy hồi-sinh hiện nay. Với tiếng Việt, bộ dữ liệu UIT-ViQuAD 2.0 [2] cung cấp nền tảng đánh giá chung theo định dạng SQuAD 2.0 [4], gồm cả câu hỏi *không có đáp án*.

Hai họ mô hình Transformer tiền huấn luyện thường được dùng cho bài toán này khác nhau ở một điểm mà môn học nhấn mạnh từ Buổi 3 — **tách từ**:

- **PhoBERT** [8] là mô hình *đơn ngữ* tiếng Việt, tiền huấn luyện trên văn bản *đã tách từ*: các âm tiết của một từ được nối bằng dấu gạch dưới (“Hà_Nội”, “thủ_đô”). Đơn vị đầu vào của nó là *từ*.
- **XLM-R** [7] là mô hình *đa ngữ* (100 ngôn ngữ), dùng SentencePiece [9] trên văn bản thô. Đơn vị đầu vào của nó là *mảnh từ con* (subword), không cần tách từ trước.

Khác biệt này không chỉ là chi tiết cài đặt. Một mô hình đọc hiểu *trích xuất* trả lời bằng cách chọn token bắt đầu và token kết thúc. Nếu đơn vị nhỏ nhất là một *từ đã tách*, mô hình không thể trả về nửa từ: khi ranh giới của đáp án vàng cắt ngang một từ ghép theo bộ tách từ, mô hình word-level *không có cách nào* khớp chính xác. Ngược lại, mô hình subword có thể mất lợi thế khi đơn vị ngữ nghĩa của tiếng Việt là từ nhiều âm tiết. Luận điểm xuyên suốt của môn học — “*phải dựa trên đặc điểm dữ liệu để chọn phương pháp*” — vì vậy có một phiên bản cụ thể, kiểm chứng được: *đặc điểm tách từ của tiếng Việt ảnh hưởng thế nào đến việc chọn giữa mô hình word-level và subword cho MRC?*

Các bảng xếp hạng MRC chỉ báo một điểm tổng EM/F1. Điểm tổng trả lời được câu hỏi “mô hình nào cao hơn” nhưng không trả lời được “mô hình sai ở đâu, và vì sao”. Đề án này đặt trọng tâm vào câu hỏi thứ hai.

1.2 Phát biểu bài toán

Cho đoạn văn $C = (c_1, \dots, c_{|C|})$ là chuỗi ký tự và câu hỏi Q . Hệ thống cần tìm cặp chỉ số ký tự (s, e) , $0 \leq s < e \leq |C|$, sao cho đoạn con $C[s:e]$ trả lời Q ; hoặc trả về chuỗi rỗng $\emptyset$ nếu C không chứa câu trả lời.

1.3 Câu hỏi nghiên cứu

Theo hướng dẫn của môn học ở Buổi 3, câu hỏi nghiên cứu được đặt ở dạng kiểm chứng được.

Giả thuyết ban đầu cho RQ1 (ghi trong đề cương trước khi chạy): PhoBERT yếu

Bảng 1. Đặc tả đầu vào, đầu ra

Thành phần	Mô tả
Đầu vào	context : đoạn văn Wikipedia tiếng Việt; question : câu hỏi tự nhiên
Đầu ra	answer : một chuỗi <i>con</i> của context , hoặc chuỗi rỗng
Loại bài toán	Trích xuất (extractive), không sinh văn bản
Dữ liệu	UIT-ViQuAD 2.0 (định dạng SQuAD 2.0)
Thang đo	Exact Match (EM), F1 trên token; cộng các phép đo chẩn đoán ở Mục 3

Bảng 2. Bốn câu hỏi nghiên cứu và phép đo trả lời từng câu

	Câu hỏi	Phép đo (Mục)
RQ1	Mô hình word-level (PhoBERT) có nhạy với ranh giới từ hơn mô hình subword (XLM-R) không?	Trần EM do biên từ; điểm trên câu lệch/không lệch biên; loại lỗi biên (4.6)
RQ2	Hiệu năng suy giảm theo độ dài ngữ cảnh và độ dài đáp án có khác nhau giữa hai họ mô hình không?	Tách nhóm theo độ dài (4.7)
RQ3	Mô hình thật sự nhận ra câu không có đáp án, hay chỉ học thiên lệch về việc từ chối?	Mức sàn “luôn từ chối”; precision/recall của hành vi từ chối (4.8)
RQ4	Transformer có vượt baseline không-neural ở <i>mọi</i> nhóm câu hỏi không?	So TF-IDF trên từng nhóm (4.9)

hơn ở các câu có ranh giới từ ghép phức tạp vì *lỗi tách từ lan truyền* vào mô hình, còn XLM-R không phụ thuộc bước này. Hai mô hình được chọn vì chúng khác nhau *đúng ở khâu tokenization* — đây là một tương phản được thiết kế, không phải so sánh ngẫu nhiên. Mục 4.6 đối chiếu giả thuyết này với số liệu.

1.4 Đối chiếu với yêu cầu môn học

Yêu cầu bắt buộc của đề án CS221 là giải một bài toán NLP có đầu vào là văn bản hoặc tiếng nói; sản phẩm là mô hình được huấn luyện (“train model thôi, mọi người không cần làm app”), kèm báo cáo và thuyết trình. Bảng 3 ánh xạ các yêu cầu và các chủ đề được nhấn mạnh trong môn học vào nội dung đề án.

Bảng 3. Đối chiếu yêu cầu và nội dung môn học với đề án

Yêu cầu / nội dung môn học	Thực hiện trong đề án	Mục
Bài toán NLP, đầu vào văn bản	Đọc hiểu trích xuất tiếng Việt	1.2
Huấn luyện mô hình	Fine-tune PhoBERT-base-v2 và XLM-R-base trên toàn bộ tập train	3.5
Buổi 3 — tiền xử lý, tách từ	Bộ tách từ cho PhoBERT; phép đo tương thích biên từ	3.4, 3.6
Buổi 3 — quy trình nghiên cứu	Câu hỏi nghiên cứu dạng kiểm chứng; giả thuyết ghi trước	1.3
Buổi 4 — TF-IDF	Baseline truy hồi câu bằng TF-IDF	3.3
Buổi 5–7 — phân tích lỗi	Phân loại lỗi tự động trên toàn bộ tập chấm; phân tích định tính	3.7, 4.10
Buổi 7 — giới hạn độ dài, từ vs âm tiết	Cửa sổ trượt; so sánh độ dài chuỗi token hai tokenizer	2.7, 4.7
Buổi 7 — chất lượng nhân	Kiểm định bộ stress-test của nhóm	4.3

1.5 Phạm vi

Đề án giới hạn trong hỏi đáp trích xuất trên *một* đoạn văn cho trước. Nằm ngoài phạm vi: hỏi đáp sinh văn bản, truy hồi đoạn văn từ kho lớn, hỏi đáp hội thoại, và dữ liệu ngoài miền Wikipedia. Mọi kết luận chỉ có giá trị trên tập validation của UIT-ViQuAD 2.0, với một seed huấn luyện và cấu hình được mô tả.

1.6 Đóng góp chính

- Đưa PhoBERT vào đọc hiểu trích xuất mà không cần Java.** PhoBERT chỉ có tokenizer “chậm”, không trả về vị trí ký tự (offset). Nhóm cài một lớp bọc tự dựng offset từ bước tách từ, được kiểm thử bằng bất biến “giải mã nhân phải ra lại đúng đáp án vàng” (Mục 3.4).
- Một phép đo chẩn đoán không cần chạy mô hình:** với mỗi câu hỏi, đo xem biên đáp án vàng có trùng biên từ của bộ tách từ không, và tính *trần EM* mà một

mô hình word-level hoàn hảo đạt được. Phép đo chạy trên toàn bộ 2.653 câu có đáp án của validation, không phải trên vài chục câu (Mục 3.6).

3. **So sánh có kiểm soát** PhoBERT và XLM-R trên cùng dữ liệu, cùng lịch học, cùng quy trình chọn epoch; chênh lệch được kiểm định bằng McNemar và bootstrap trên từng câu thay vì so hai con số tổng (Mục 3.9).
4. **Kiểm định bộ stress-test của chính nhóm** trước khi dùng nó. Kết quả là một phát hiện âm tính quan trọng: chỉ 15/120 câu có đáp án chứa đáp án trong ngữ cảnh; nguyên nhân được truy tới hai hàm cụ thể trong mã sinh dữ liệu. Bộ này vì vậy *không* được dùng để xếp hạng mô hình (Mục 4.3).
5. **Tái lập được**: mọi con số trong báo cáo được sinh từ tệp `results/*.json` kèm commit; chính các bảng và con số trong văn bản của báo cáo này được sinh tự động từ các tệp đó (Phụ lục D).

1.7 Cấu trúc báo cáo

Mục 2 trình bày cơ sở lý thuyết. Mục 3 mô tả phương pháp: pipeline, cách đưa PhoBERT vào bài toán trích xuất, các phép đo chẩn đoán và giao thức thực nghiệm. Mục 4 trình bày kết quả và trả lời lần lượt bốn câu hỏi nghiên cứu. Mục 5 kết luận và nêu giới hạn. Phụ lục cung cấp hướng dẫn tái lập và bảng truy vết số liệu.

2 Cơ sở lý thuyết

2.1 Bài toán đọc hiểu máy

2.1.1 Phân loại

Đọc hiểu máy là nhóm bài toán yêu cầu mô hình trả lời câu hỏi dựa trên một văn bản cho trước. Theo dạng đầu ra, các bộ dữ liệu MRC thường được chia thành bốn loại (Bảng 4). Đề án thuộc loại *trích xuất đoạn* (span extraction).

Bảng 4. Các dạng bài toán đọc hiểu máy theo dạng đầu ra

Dạng	Đầu ra	Đặc điểm đánh giá
Điền khuyết (cloze)	Một từ/cụm từ bị che trong văn bản	So khớp chính xác, dễ tự động hoá
Trắc nghiệm	Chọn một trong k phương án	Accuracy
Trích xuất đoạn	Đoạn con liên tục của văn bản	EM, F1 trên token
Tự do (free-form)	Văn bản sinh mới	Khó đánh giá; cần BLEU/ROUGE hoặc người chấm

Dạng trích xuất có hai ưu điểm quan trọng cho một đề án có yêu cầu nghiêm ngặt về đánh giá: không gian đầu ra bị chặn trong văn bản nên metric so khớp chuỗi có ý nghĩa, và câu trả lời luôn kiểm chứng được bằng cách chỉ ra vị trí của nó trong nguồn.

2.1.2 SQuAD và vấn đề câu hỏi không có đáp án

SQuAD 1.1 [3] gồm hơn 100.000 câu hỏi do con người đặt trên các bài viết Wikipedia; mọi câu hỏi đều có đáp án trong đoạn văn. Hệ quả là mô hình có thể đạt điểm cao bằng cách luôn chọn đoạn “trông giống câu trả lời nhất” mà không cần hiểu đoạn văn có thực sự trả lời câu hỏi hay không. SQuAD 2.0 [4] khắc phục điều này bằng cách bổ sung hơn 50.000 câu hỏi *không có đáp án* được người viết cố tình tạo cho giống câu hỏi có đáp án — ví dụ thay một thực thể trong câu hỏi bằng một thực thể khác. Mô hình phải học thêm kỹ năng *từ chối*: trả về chuỗi rỗng khi đoạn văn không chứa thông tin cần thiết.

2.1.3 UIT-ViQuAD và UIT-ViQuAD 2.0

UIT-ViQuAD [1] là bộ dữ liệu MRC tiếng Việt quy mô lớn, gồm hơn 23.000 cặp câu hỏi–câu trả lời do con người tạo trên 5.109 đoạn văn thuộc 174 bài viết Wikipedia tiếng Việt, theo định dạng SQuAD. UIT-ViQuAD 2.0 được công bố cho cuộc thi VLSP 2021 ViMRC [2], bổ sung câu hỏi không có đáp án theo tinh thần SQuAD 2.0. Mỗi câu hỏi không có đáp án có trường `is_impossible = True` và có thể kèm `plausible_answers` — một đáp án “nghe hợp lý nhưng sai”. Trường này *không bao giờ* được dùng làm nhãn đúng; dùng nhầm sẽ biến câu không có đáp án thành câu có đáp án và làm sai metric một cách âm thầm. Thống kê chi tiết của phiên bản dữ liệu dùng trong đề án được trình bày ở Mục 4.2.

2.2 Tách từ tiếng Việt

Tiếng Việt là ngôn ngữ đơn lập: chữ viết tách theo *âm tiết* bằng khoảng trắng, còn một *từ* có thể gồm nhiều âm tiết (“Hà Nội”, “thủ đô”, “chủ nghĩa xã hội”). Khoảng trắng vì vậy không đánh dấu ranh giới từ, và **tách từ** (word segmentation) — nội dung của Buổi 3 — là bước tiền xử lý đặc thù: xác định âm tiết nào ghép thành từ. Các bộ tách từ phổ biến gồm RDRSegmenter trong VnCoreNLP [16] (dựa trên luật được học theo phương pháp Ripple Down Rules, cần Java) và pyvi [17] (mô hình CRF gán nhãn chuỗi, thuần Python). Đầu ra quy ước nối âm tiết trong từ bằng dấu gạch dưới:

Hà Nội là thủ đô của nước Việt Nam → Hà_Nội là thủ_đô của nước Việt_Nam

Ba hệ quả cho bài toán đọc hiểu trích xuất:

1. **Tách từ là một mô hình, và nó có lỗi.** Bộ tách từ khác nhau cho kết quả khác nhau trên cùng câu, đặc biệt với tên riêng nước ngoài, từ viết tắt và dấu câu. Một mô hình dùng đầu vào đã tách từ *thừa hưởng* các lỗi đó.
2. **Đơn vị đầu ra bị giới hạn bởi đơn vị đầu vào.** Mô hình trích xuất trả lời bằng cách chọn token đầu và token cuối. Nếu token là từ đã tách, đáp án chỉ có thể là một dãy từ nguyên vẹn (Mục 3.6).
3. **Dấu thanh là một phần của chữ.** “hoà” và “hoa” là hai từ khác nhau; mọi bước chuẩn hoá hoặc giải mã làm mất dấu sẽ âm thầm làm sai kết quả.

2.3 Truy hồi câu bằng TF-IDF

TF-IDF biểu diễn một văn bản d thành vector trọng số trên từ vựng. Với `TfidfVectorizer` của scikit-learn [11] ở cấu hình mặc định, trọng số của term t trong văn bản d thuộc tập gồm N văn bản là

$$w_{t,d} = \text{tf}(t, d) \cdot \left(\ln \frac{1 + N}{1 + \text{df}(t)} + 1 \right), \quad (1)$$

trong đó $\text{tf}(t, d)$ là số lần t xuất hiện trong d và $\text{df}(t)$ là số văn bản chứa t . Mỗi vector sau đó được chuẩn hoá L2. Độ tương đồng giữa câu hỏi q và một câu s là cosine của hai vector:

$$\text{sim}(q, s) = \frac{\mathbf{w}_q \cdot \mathbf{w}_s}{\|\mathbf{w}_q\| \|\mathbf{w}_s\|}. \quad (2)$$

Baseline truy hồi chọn câu $s^* = \arg \max_s \text{sim}(q, s)$ trong đoạn văn làm câu trả lời. Phương pháp này trả lời câu hỏi khoa học: *bài toán có giải được bằng so khớp từ khoá thuần tuý không?* Vì nó trả về cả một câu trong khi đáp án vàng thường là cụm vài từ, có thể dự đoán trước rằng F1 của nó ở mức trung bình còn EM gần bằng 0.

2.4 Kiến trúc Transformer

Transformer [5] thay thế mạng hồi quy bằng cơ chế *tự chú ý* (self-attention), cho phép mỗi token trực tiếp tổng hợp thông tin từ mọi token khác trong chuỗi. Với ma trận truy vấn Q , khoá K và giá trị V được chiếu tuyến tính từ biểu diễn đầu vào, attention được tính bằng

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (3)$$

trong đó d_k là số chiều của khoá; phép chia cho $\sqrt{d_k}$ giữ cho tích vô hướng không quá lớn làm softmax bão hoà. *Multi-head attention* chạy h phép attention song song trên các không gian chiếu khác nhau rồi ghép kết quả:

$$\begin{aligned} \text{MultiHead}(X) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \\ \text{head}_i &= \text{Attention}(XW_i^Q, XW_i^K, XW_i^V). \end{aligned} \quad (4)$$

Mỗi lớp encoder gồm một khối multi-head attention và một mạng truyền thẳng hai lớp, mỗi khối có kết nối tắt (residual) và chuẩn hoá lớp (layer normalization). Vì attention không có khái niệm thứ tự, vị trí được mã hoá bằng *position embedding* với số vị trí tối đa cố định — 512 với XLM-R nhưng chỉ 258 với PhoBERT (256 token hữu dụng). Đây là lý do văn bản dài phải được cắt thành nhiều cửa sổ (Mục 2.7).

2.5 Mô hình ngôn ngữ tiền huấn luyện

2.5.1 BERT và RoBERTa

BERT [6] là chồng các lớp encoder Transformer được tiền huấn luyện trên văn bản không nhãn bằng *masked language modeling* (dự đoán token bị che) và *next sentence prediction*; biểu diễn thu được có tính hai chiều và được fine-tune cho tác vụ đích bằng cách thêm một lớp đầu ra nhỏ. RoBERTa [14] giữ kiến trúc nhưng bỏ next sentence prediction, dùng nhiều dữ liệu hơn, batch lớn hơn và che token động. Cả hai mô hình của đề án đều theo công thức RoBERTa, cỡ *base*: 12 lớp, 768 chiều ẩn, 12 đầu attention. Vì kiến trúc encoder giống nhau, khác biệt giữa chúng nằm ở **dữ liệu tiền huấn luyện** và **đơn vị token** — đúng hai biến đề án muốn tách ra.

2.5.2 PhoBERT — đơn ngữ, word-level

PhoBERT [8] là RoBERTa đơn ngữ tiếng Việt, tiền huấn luyện trên khoảng 20 GB văn bản Wikipedia và tin tức *đã tách từ bằng RDRSegmenter*, với bộ mã hoá BPE [15] áp dụng *trên từng từ đã tách*. Bản `vinai/phobert-base-v2` dùng trong đề án có từ vựng 64.001 token và **258 vị trí** (256 token hữu dụng). Hai ràng buộc kỹ thuật định hình cách dùng nó:

- Đầu vào *phải* được tách từ trước; đưa văn bản thô vào vẫn chạy nhưng lệch khỏi phân phối tiền huấn luyện.
- Trên HuggingFace, PhoBERT chỉ có tokenizer dạng Python (“slow”), *không* trả về *offset mapping* — vị trí ký tự của mỗi token trong văn bản gốc. Thiếu thông tin này thì không thể biến chỉ số token của đáp án thành chuỗi ký tự trong văn bản (Mục 2.5.4). Đề án giải quyết ràng buộc này ở Mục 3.4.

2.5.3 XLM-RoBERTa — đa ngữ, subword

XLM-R [7] áp dụng công thức RoBERTa cho 100 ngôn ngữ trên khoảng 2,5 TB văn bản CommonCrawl, với từ vựng SentencePiece [9] khoảng 250.000 token dùng chung cho mọi ngôn ngữ, và 514 vị trí (512 hữu dụng). SentencePiece làm việc trên văn bản thô, coi khoảng trắng là một ký tự (“_” ở đầu mảnh), nên *không cần* bộ tách từ và mọi mảnh đều có vị trí ký tự xác định. Bản `FacebookAI/xlm-roberta-base` dùng trong đề án *chưa* được fine-tune cho hỏi đáp — nó được huấn luyện trên ViQuAD với cùng quy trình như PhoBERT, để so sánh là công bằng.

2.5.4 Offset mapping

Mô hình dự đoán vị trí đáp án theo *chỉ số token*, trong khi đáp án cần trả về là *chuỗi ký tự* trong văn bản gốc. Cầu nối là **offset mapping**: với mỗi token, cặp (start_char, end_char) trong chuỗi gốc. Cách thay thế — ghép lại chuỗi bằng `tokenizer.decode()` — không đảm bảo khôi phục đúng dấu tiếng Việt và khoảng trắng gốc, và với PhoBERT còn trả về dấu gạch dưới của bước tách từ (“Hà_Nội”), khiến một đáp án đúng bị chấm EM 0 (Mục 3.4).

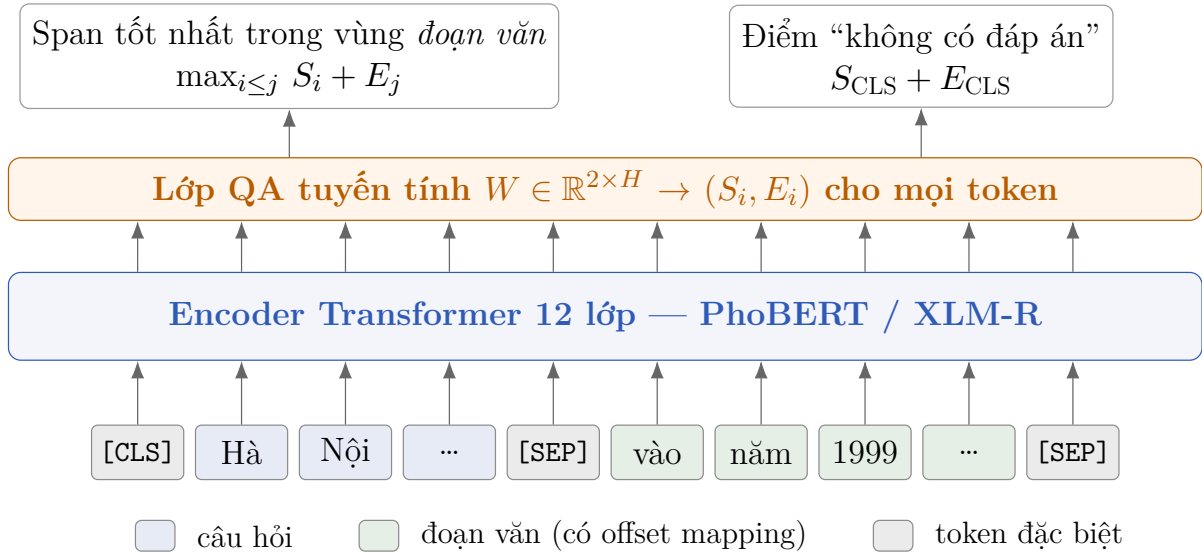
2.6 Mô hình hỏi đáp trích xuất trên encoder

2.6.1 Đầu vào và lớp QA

Câu hỏi và đoạn văn được ghép thành một chuỗi $\langle s \rangle$ câu hỏi $\langle /s \rangle \langle /s \rangle$ đoạn văn $\langle /s \rangle$ (định dạng RoBERTa, dùng cho cả hai mô hình; ở đây viết gọn là [CLS] / [SEP]) và đưa qua encoder, thu được biểu diễn $\mathbf{h}_i \in \mathbb{R}^H$ cho mỗi token i . Lớp QA là một phép chiếu tuyến tính $W \in \mathbb{R}^{2 \times H}$ sinh hai điểm cho mỗi token: điểm bắt đầu S_i và điểm kết thúc E_i (Hình 1). Xác suất token i là vị trí bắt đầu là

$$P_{\text{start}}(i) = \frac{\exp(S_i)}{\sum_j \exp(S_j)}, \quad (5)$$

và tương tự cho P_{end} .



Hình 1. Kiến trúc hỏi đáp trích xuất: encoder dùng chung, lớp QA sinh điểm bắt đầu/kết thúc cho mỗi token; token [CLS] đại diện cho lựa chọn “không có đáp án”

2.6.2 Hàm mất mát

Với nhãn vị trí bắt đầu y_s và kết thúc y_e (chỉ số token), mô hình được huấn luyện bằng trung bình hai hàm cross-entropy:

$$\mathcal{L} = -\frac{1}{2} (\log P_{\text{start}}(y_s) + \log P_{\text{end}}(y_e)). \quad (6)$$

Theo quy ước SQuAD 2.0, với câu không có đáp án — và với cửa sổ không chứa trọn đáp án — nhãn được gán về token [CLS]: $y_s = y_e = \text{idx}_{\text{CLS}}$. Đây là cách mô hình học biểu đạt “ở đây không có đáp án”. Với UIT-ViQuAD 2.0, nhánh này chiếm khoảng một phần ba dữ liệu huấn luyện, nên sai sót trong gán nhãn không phải trường hợp biên hiếm gặp.

2.6.3 Suy luận: chọn span và quyết định không trả lời

Khi suy luận, span được chọn là cặp (i, j) có tổng điểm lớn nhất trong các ràng buộc:

$$(i^*, j^*) = \arg \max_{\substack{i \leq j, j-i+1 \leq L_{\max} \\ i, j \in \text{đoạn văn}}} S_i + E_j, \quad (7)$$

trong đó $L_{\max}$ là độ dài đáp án tối đa tính theo token. Ràng buộc “ i, j thuộc đoạn văn” loại các token của câu hỏi và token đặc biệt; ràng buộc $i \leq j$ loại span đảo ngược — hai lỗi thường gặp khi lấy $\arg \max$ độc lập cho S và E . Mô hình trả về chuỗi rỗng nếu

$$S_{i^*} + E_{j^*} \leq S_{\text{CLS}} + E_{\text{CLS}} + \tau, \quad (8)$$

với τ là *ngưỡng không trả lời*. Tăng τ làm mô hình dè dặt hơn (ít trả lời sai trên câu không có đáp án, nhưng bỏ sót câu có đáp án); giảm τ có tác dụng ngược lại. Trong mọi thực nghiệm chính, $\tau = 0$.

2.7 Cửa sổ trượt cho văn bản dài

Khi độ dài câu hỏi cộng đoạn văn vượt giới hạn `max_length` token, đoạn văn được cắt thành các cửa sổ chồng lấp. Gọi N là số token của đoạn văn, $|Q|$ là số token của câu hỏi và n_{sp} là số token đặc biệt. Ngân sách token cho đoạn văn trong mỗi cửa sổ và bước trượt là

$$B = \text{max_length} - |Q| - n_{\text{sp}}, \quad \text{step} = B - \text{doc_stride}, \quad (9)$$

và số cửa sổ cần thiết là

$$n_{\text{win}} = 1 + \left\lceil \frac{\max(0, N - B)}{\text{step}} \right\rceil. \quad (10)$$

Phần chồng lấp `doc_stride` đảm bảo đáp án nằm gần ranh giới cửa sổ vẫn xuất hiện trọn vẹn trong ít nhất một cửa sổ. Khi suy luận, span tốt nhất của từng cửa sổ được so sánh theo điểm $S_i + E_j$ để chọn đáp án cuối; nếu mọi cửa sổ đều chọn “không có đáp án”, kết quả là chuỗi rỗng.

2.8 Tối ưu hoá khi fine-tune

AdamW. Đề án dùng AdamW [10], biến thể của Adam tách rời weight decay khỏi cập nhật gradient:

$$\theta_t = \theta_{t-1} - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_{t-1} \right), \quad (11)$$

với $\hat{m}_t, \hat{v}_t$ là ước lượng moment bậc một và hai đã hiệu chỉnh độ lệch, và λ là hệ số weight decay.

Lịch học tuyến tính có warmup. Tốc độ học tăng tuyến tính từ 0 trong T_w bước đầu rồi giảm tuyến tính về 0 tại bước cuối T :

$$\eta_t = \begin{cases} \eta_{\max} \cdot t/T_w, & t < T_w, \\ \eta_{\max} \cdot (T - t)/(T - T_w), & t \geq T_w. \end{cases} \quad (12)$$

Warmup tránh việc gradient lớn ở những bước đầu — khi lớp QA còn khởi tạo ngẫu nhiên — phá hỏng biểu diễn đã tiền huấn luyện.

Tích lũy gradient và cắt gradient. Tích lũy gradient qua k batch trước mỗi lần cập nhật cho batch hiệu dụng lớn hơn bộ nhớ cho phép. Khi đó một “bước” của lịch học là một lần cập nhật tham số, không phải một batch — đếm nhằm đơn vị này làm tốc độ học giảm về 0 quá sớm. Chuẩn gradient được cắt ở ngưỡng 1,0 để ổn định huấn luyện.

2.9 Thang đo đánh giá

2.9.1 Chuẩn hoá chuỗi

Trước khi so khớp, dự đoán p và đáp án vàng g được chuẩn hoá bởi hàm $\text{norm}(\cdot)$: chuyển chữ thường, bỏ ký tự dấu câu Unicode, gộp khoảng trắng. Hàm tách token $\text{tok}(\cdot)$ tách chuỗi đã chuẩn hoá theo khoảng trắng thành danh sách âm tiết. Metric được tính trên *âm tiết*, không trên từ đã tách: nếu tính trên từ, lỗi của bộ tách từ sẽ đi vào chính con số đánh giá, và hai mô hình dùng hai cách tách khác nhau sẽ bị chấm bằng hai thước đo khác nhau.

2.9.2 Exact Match

$$\text{EM}(p, g) = \mathbb{1}[\text{norm}(p) = \text{norm}(g)]. \quad (13)$$

2.9.3 F1 trên token

Gọi $T_p = \text{tok}(p)$ và $T_g = \text{tok}(g)$ là *địa tập* token (token lặp được đếm đúng số lần). Số token chung là $c = |T_p \cap T_g|$. Khi đó

$$P = \frac{c}{|T_p|}, \quad R = \frac{c}{|T_g|}, \quad F1(p, g) = \frac{2PR}{P+R}, \quad (14)$$

với quy ước $F1 = 0$ khi $c = 0$. Nếu một trong hai bên rỗng, $F1 = 1$ khi và chỉ khi *cả hai* cùng rỗng — không có điểm bán phần.

2.9.4 Nhiều đáp án vàng và câu không có đáp án

Một câu hỏi có thể có nhiều đáp án vàng $G = \{g_1, \dots, g_m\}$; điểm được lấy theo đáp án thuận lợi nhất:

$$\text{score}(p, G) = \max_{g \in G} \text{metric}(p, g). \quad (15)$$

Với câu không có đáp án, $G = \emptyset$ và p được so với chuỗi rỗng: chỉ dự đoán rỗng được điểm 1. Hệ quả quan trọng: **trên câu không có đáp án, EM và F1 luôn bằng nhau**. Điểm tổng hợp trên tập n câu là trung bình nhân 100:

$$\text{EM}_{\text{tổng}} = \frac{100}{n} \sum_{k=1}^n \text{score}_{\text{EM}}(p_k, G_k), \quad (16)$$

và tương tự cho F1. Vì vậy điểm tổng *trộn hai kỹ năng*: tìm đúng span trên câu có đáp án và biết từ chối trên câu không có đáp án. Báo cáo luôn tách riêng hai nhóm này.

2.10 Đánh giá chẩn đoán

Một điểm tổng trên tập test trả lời “mô hình nào tốt hơn” nhưng không nói mô hình *sai ở đâu*. Nhiều công trình đã đề xuất đánh giá có cấu trúc: *stress test* cho suy luận ngôn ngữ tự nhiên [19], câu hỏi đối kháng cho SQuAD [18], *contrast set* — biến đổi nhỏ có kiểm soát trên mẫu test [20] — và CheckList, kiểm thử hành vi theo năng lực ngôn ngữ [21]. Điểm chung: bộ kiểm thử cô lập *một* hiện tượng để thất bại quy được về nguyên nhân. Bowman và Dahl [22] lưu ý rằng bộ chẩn đoán không thay thế benchmark: chúng có giá trị khi *nhãn đúng* và *đo đúng thứ chúng tuyên bố đo*. Điều kiện thứ hai là lý do đồ án kiểm định bộ stress-test của nhóm trước khi dùng (Mục 4.3).

Hai cách tạo nhóm chẩn đoán được dùng trong đồ án:

- **Bộ dữ liệu xây riêng** cho từng hiện tượng (bộ stress-test E1–E5): kiểm soát tốt hiện tượng nhưng nhỏ, và phụ thuộc hoàn toàn vào chất lượng của quy trình tạo dữ liệu.
- **Chia nhóm tập chấm có sẵn** theo một thuộc tính đo tự động được (độ dài, loại câu hỏi, tương thích biên từ): cỡ mẫu lớn và nhãn do người gán, nhưng hiện tượng không được cô lập hoàn toàn.

2.11 So sánh hai mô hình trên cùng tập chấm

Hai mô hình được chấm trên *cùng* các câu hỏi, nên phép so sánh đúng là so sánh *cặp*, không phải so hai con số tổng như hai mẫu độc lập [23].

Kiểm định McNemar. Với EM là biến nhị phân, mỗi câu rơi vào một trong bốn ô: cả hai đúng, chỉ A đúng (b câu), chỉ B đúng (c câu), cả hai sai. Chỉ các câu bất đồng mang thông tin về chênh lệch. Dưới giả thuyết không (hai mô hình như nhau), $b \sim \text{Binomial}(b+c, \frac{1}{2})$, cho p-value chính xác hai phía [24]

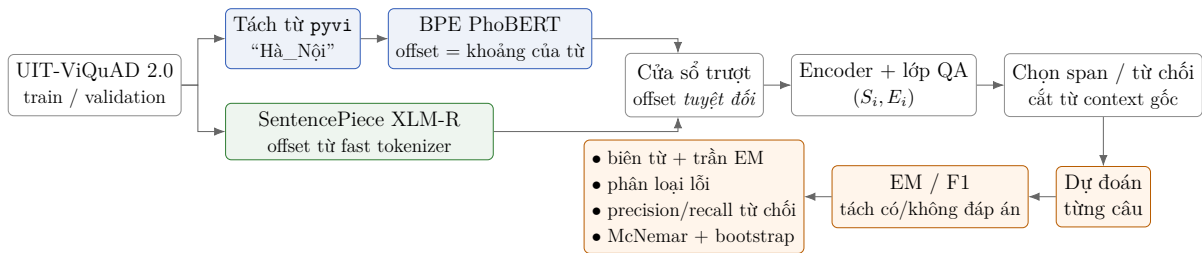
$$p = \min \left(1, 2 \sum_{k=0}^{\min(b,c)} \binom{b+c}{k} 2^{-(b+c)} \right). \quad (17)$$

Bootstrap. Khoảng tin cậy 95% cho chênh lệch EM được ước lượng bằng bootstrap phân vị [25]: lấy mẫu có hoàn lại n câu, tính chênh lệch EM trên mẫu đó, lặp 2.000 lần và lấy phân vị 2,5% và 97,5%. Khoảng không chứa 0 nghĩa là chênh lệch nhất quán qua các cách lấy mẫu câu hỏi. Cần lưu ý: cả hai phép này đo độ bất định do *chọn câu hỏi*, *không* đo độ bất định do *seed huấn luyện* — đề án chỉ huấn luyện một seed cho mỗi mô hình (Mục 5.2).

3 Phương pháp

3.1 Tổng quan

Hình 2 là toàn bộ quy trình. Bốn hệ thống được chấm bằng *cùng* một harness đánh giá: hai baseline không học (luôn từ chối; TF-IDF) và hai Transformer được fine-tune (PhoBERT, XLM-R). Điểm khác biệt duy nhất giữa hai nhánh Transformer là bước biến văn bản thành token — đúng biến mà câu hỏi nghiên cứu RQ1 muốn đo. Mọi hệ thống ghi ra dự đoán *từng câu*, để các phép đo chẩn đoán ở Mục 3.6–3.9 ghép được với từng câu hỏi thay vì chỉ với con số tổng.



Hình 2. Quy trình. Nhánh xanh dương (PhoBERT) và xanh lá (XLM-R) chỉ khác nhau ở bước tạo token và offset; mọi bước sau dùng chung mã

Tái sử dụng mã nguồn. Tầng dữ liệu, metric, cửa sổ trượt, vòng lặp huấn luyện và harness đánh giá (`src/mrc/`) được kế thừa từ đề án môn CS116 của cùng nhóm tác giả, vốn đã có bộ kiểm thử. Đề án CS221 bổ sung phần phục vụ câu hỏi nghiên cứu riêng: lớp tokenizer cho PhoBERT, phép đo biên từ, kiểm định stress-test, các phép đo chẩn đoán và giao thức chọn epoch trên tập dev tách từ train. Hai đề án trả lời hai câu hỏi khác nhau trên hai tập mô hình khác nhau; không con số nào của đề án CS116 được dùng lại trong báo cáo này. Khung mã ban đầu của chính đề án CS221 (thư mục `legacy/`) *không* được dùng để sinh kết quả vì các lỗi được nêu ở Phụ lục B.

3.2 Giao thức dữ liệu

- **Test chính thức của UIT-ViQuAD 2.0 là tập ẩn nhãn.** Toàn bộ 7.301 câu có đáp án rỗng nhưng `is_impossible = False`; không chấm được. Harness từ chối chấm các câu như vậy bằng một kiểm tra chặn (`assert_gradeable`).
- **Validation chính thức (3.814 câu) đóng vai tập test của đề án,** được chấm *đúng một lần* cho mỗi hệ thống, trên toàn bộ tập, không lấy mẫu con.

- **Chọn epoch trên tập dev tách từ train:** 205 đoạn văn (1.405 câu, 5% số đoạn văn) được tách khỏi train *theo đoạn văn*, để không câu hỏi nào của cùng một đoạn văn nằm ở cả hai phía. Mô hình được huấn luyện trên phần còn lại (27.049 câu). Như vậy validation không ảnh hưởng tới bất kỳ quyết định nào trước khi chấm.
- **Chống rò rỉ:** trước khi huấn luyện, chương trình khẳng định (`assert`) rằng ba tập train/dev/validation không có đoạn văn chung; vi phạm làm dừng chương trình.
- **Bộ stress-test chỉ dùng để đánh giá**, không bao giờ trộn vào dữ liệu huấn luyện (khung mã cũ làm điều này — Phụ lục B).

3.3 Hai baseline

Luôn từ chối. Trả về chuỗi rỗng cho mọi câu. Theo quy ước chấm SQuAD 2.0, hệ thống này đạt điểm bằng đúng tỉ lệ câu không có đáp án của tập chấm. Nó không phải một mô hình mà là một *thước đo*: mọi con số khác phải được đọc so với mức sàn này, và nó làm lộ ra những nhóm câu hỏi mà điểm cao không chứng minh năng lực gì (Mục 4.8).

TF-IDF. Truy hồi câu bằng TF-IDF (Mục 2.3): tách đoạn văn thành câu, vector hoá câu hỏi và các câu bằng `TfidfVectorizer` (n-gram 1–2), trả về câu có cosine lớn nhất. Baseline này không bao giờ từ chối và luôn trả về *cả một câu*.

3.4 Đưa PhoBERT vào bài toán trích xuất

PhoBERT cần đầu vào đã tách từ nhưng không cung cấp offset. Lớp `SegmentedTokenizer` tự dựng offset theo Thuật toán 1: tách từ bằng `pyvi`, căn từng từ về khoảng ký tự của nó trong văn bản gốc, rồi mã hoá BPE *từng từ*; mọi mảnh BPE của một từ mang chung khoảng ký tự của cả từ. Lớp này cài đúng phần giao diện mà mã cửa sổ trượt và mã suy luận sử dụng, nên PhoBERT và XLM-R đi qua *cùng* một đường mã từ đó trở đi.

Hai hệ quả của thiết kế này được dùng trực tiếp trong phân tích:

1. **Đáp án luôn được cắt từ văn bản gốc**, nên không bao giờ chứa dấu gạch dưới của bước tách từ. Cách ngây thơ — giải mã token bằng `decode()` — trả về “Hà_Nội”; vì bước chuẩn hoá của metric xoá dấu câu, chuỗi này thành “hànoị” và một đáp án *đúng* bị chấm EM 0, F1 0. Lỗi này tồn tại trong khung mã ban đầu và được kiểm thử hồi quy.
2. **PhoBERT chỉ trả lời được bằng dãy từ nguyên vẹn** — đúng tính chất mà phép đo ở Mục 3.6 khai thác.

Tính đúng đắn được kiểm thử bằng các bất biến: mỗi offset cắt ra đúng từ gốc (thay “_” bằng khoảng trắng); nhãn huấn luyện được giải mã ra lại đúng đáp án vàng khi đáp án khớp biên từ; và bị *mở rộng* đúng tới từ bao quanh khi đáp án cắt ngang từ. Trên validation, các từ đã căn phủ 100,00% ký tự không phải khoảng trắng của mọi đoạn văn — không ký tự nào bị bỏ sót.

Thuật toán 1: Dựng offset cho PhoBERT từ bước tách từ**Đầu vào:** văn bản gốc T ; bộ tách từ SEG ; tokenizer BPE**Đầu ra:** danh sách ids , danh sách offsets (khoảng ký tự trong T)

```

1  $p \leftarrow 0$ ;
2 với mỗi từ  $w$  trong  $\text{SEG}(T)$  thực hiện
3    $s \leftarrow \perp$ ;
4   với mỗi âm tiết  $a$  trong  $w$  tách theo “_” thực hiện
5      $p \leftarrow$  vị trí đầu tiên  $\geq p$  trong  $T$  bắt đầu bằng  $a$ , bỏ qua khoảng trắng;
6     nếu không tìm thấy thì
7       bỏ qua;
8     nếu  $s = \perp$  thì
9        $s \leftarrow p$ ;
10     $p \leftarrow p + |a|$ ;
11  với mỗi mảnh BPE  $b$  của  $w$  thực hiện
12     $\mid$  thêm  $\text{id}(b)$  vào  $\text{ids}$ ; thêm  $(s, p)$  vào  $\text{offsets}$ ;
13 trả về  $\text{ids}$ ,  $\text{offsets}$ ;
```

Bộ tách từ. PhoBERT được tiền huấn luyện với RDRSegmenter (VnCoreNLP), cần Java. Đồ án dùng `pyvi`, thuần Python, để pipeline chạy được trên máy không có Java. Hai bộ tách từ không trùng nhau hoàn toàn; đây là một nguồn lệch phân phối được nêu trong giới hạn (Mục 5.2) và là một biến đáng thử trong hướng phát triển.

3.5 Fine-tuning

Hai mô hình được huấn luyện bằng *cùng* vòng lặp, cùng lịch học và cùng seed (Bảng 5); chỉ các tham số buộc phải khác do kiến trúc được thay đổi. PhoBERT chỉ có 256 vị trí hữu dụng nên `max_length` = 256 thay vì 384; phần chồng lấp giữa hai cửa sổ được giảm tương ứng. Độ dài đáp án tối đa được đặt 64 token cho cả hai, lớn hơn phân vị 95 của độ dài đáp án vàng theo cả hai tokenizer (đo trên 400 câu train: 30 token PhoBERT, 38 token XLM-R), để tham số này không trở thành một yếu tố gây nhiễu.

Không có tìm kiếm siêu tham số: tốc độ học $3 \cdot 10^{-5}$ và lịch học là giá trị phổ biến cho mô hình cỡ *base* trên SQuAD. Đây là lựa chọn có chủ đích — với một lần chạy mỗi mô hình, tìm kiếm siêu tham số trên dev rồi báo cáo trên cùng dev sẽ thổi phồng kết quả — nhưng cũng có nghĩa là cả hai mô hình có thể chưa đạt điểm tốt nhất của chúng.

3.6 Phép đo tương thích biên từ

Đây là công cụ chẩn đoán trung tâm cho RQ1, và nó *không cần chạy mô hình*. Gọi $\mathcal{W} = \{(a_k, b_k)\}$ là tập khoảng ký tự của các từ mà bộ tách từ sinh ra trên đoạn văn, và $[s, e]$ là khoảng ký tự của một đáp án vàng. Đáp án **khớp biên từ** (*aligned*) nếu không

Bảng 5. Siêu tham số huấn luyện

Tham số	PhoBERT-base-v2	XLM-R-base
Checkpoint	vinai/phobert-base-v2	FacebookAI/xlm-roberta-base
Tiền xử lý	tách từ pyvi	không
max_length / chồng lấp cửa sổ	256 / 96	384 / 128
Độ dài đáp án tối đa (token)	64	64
Số epoch tối đa	3	3
Batch $\times$ tích lũy gradient	$12 \times 2 = 24$	$12 \times 2 = 24$
Tốc độ học, warmup, weight decay	$3 \cdot 10^{-5}$; 10%; 0,01	$3 \cdot 10^{-5}$; 10%; 0,01
Tối ưu, cắt gradient	AdamW; 1,0	AdamW; 1,0
Chọn epoch	F1 cao nhất trên dev (500 câu)	F1 cao nhất trên dev (500 câu)
Ngưỡng không trả lời τ	0	0
Seed	42	42

Nguồn: trường config trong `results/training_curve_{phobert,xlmr}.json`.

đầu nào của nó nằm *bên trong* một từ:

$$\text{aligned}(s, e) \iff \nexists (a, b) \in \mathcal{W} : a < s < b \vee a < e < b. \quad (18)$$

Nếu không khớp, span nguyên-từ tốt nhất mà một mô hình word-level trả về được là span nhỏ nhất bao đáp án:

$$\hat{s} = \min\{a : (a, b) \in \mathcal{W}, a < e, b > s\}, \quad \hat{e} = \max\{b : (a, b) \in \mathcal{W}, a < e, b > s\}, \quad (19)$$

và **trần EM** của câu đó là $\max_{g \in G} \text{EM}(C[\hat{s}:\hat{e}], g)$. Với câu có nhiều đáp án vàng, câu được tính là khớp biên nếu *ít nhất một* đáp án khớp. Trung bình trần EM trên toàn tập là điểm EM cao nhất mà một mô hình word-level *hoàn hảo* có thể đạt, chỉ tính riêng giới hạn do đơn vị đầu ra.

Phép đo này tách được hai kênh mà giả thuyết RQ1 gộp làm một: (i) kênh *đơn vị đầu ra* — PhoBERT không thể trả lời nửa từ, đo được chính xác và không cần mô hình; và (ii) kênh *biểu diễn* — lỗi tách từ làm sai biểu diễn đầu vào, chỉ đo được gián tiếp qua hiệu năng. Nếu kênh (i) chỉ giải thích được một phần nhỏ khoảng cách giữa hai mô hình, phần còn lại phải đến từ nơi khác.

3.7 Phân loại lỗi tự động

Mỗi dự đoán sai được xếp vào đúng một trong sáu loại (Bảng 6), định nghĩa bằng quan hệ giữa đa tập token của dự đoán và của đáp án vàng sau chuẩn hoá. Với nhiều đáp án vàng, dùng đáp án có F1 cao nhất với dự đoán. Vì định nghĩa không cần người gán, phép phân loại chạy được trên *toàn bộ* tập chấm và tái lập được.

Ba loại “biên” là lỗi *đã tìm đúng vùng nhưng sai ranh giới* — loại lỗi mà giả thuyết RQ1 dự đoán PhoBERT mắc nhiều hơn. “Sai vị trí” là lỗi hiểu; “từ chối sai” và “trả lời sai” là lỗi của quyết định có trả lời hay không.

Bảng 6. Sáu loại lỗi

Loại	Điều kiện	Ví dụ (vàng → dự đoán)
Từ chối sai	Có đáp án, dự đoán rỗng	“Hà Nội” → $\emptyset$
Trả lời sai	Không có đáp án, dự đoán khác rỗng	$\emptyset$ → “năm 1945”
Biên thừa	Dự đoán <i>chứa</i> đáp án vàng	“Hà Nội” → “thành phố Hà Nội”
Biên thiếu	Dự đoán <i>nằm trong</i> đáp án vàng	“thành phố Hà Nội” → “Hà Nội”
Biên lệch	Có token chung, lệch cả hai phía	“Hà Nội là” → “Nội là thủ đô”
Sai vị trí	Không có token chung	“Hà Nội” → “Paris”

3.8 Hành vi từ chối như một bộ phân loại

Trên SQuAD 2.0, điểm tổng trộn hai kỹ năng. Để tách kỹ năng thứ hai, việc “trả về rỗng” được coi là một bộ phân loại nhị phân dự đoán “câu này không có đáp án”, với precision (trong những lần từ chối, bao nhiêu lần đúng) và recall (trong những câu không có đáp án, bao nhiêu câu được từ chối). Hệ thống luôn từ chối có recall 100% và precision bằng tỉ lệ câu không có đáp án — một mô hình “biết từ chối” thật phải có precision cao hơn hẳn mức đó.

3.9 So sánh cặp PhoBERT và XLM-R

Chênh lệch EM giữa hai mô hình được kiểm định bằng McNemar chính xác (công thức 17) trên các câu mà hai mô hình bất đồng, kèm khoảng tin cậy bootstrap 95% (2.000 lần lấy mẫu, seed 42) — trên toàn tập và trên từng nhóm: câu có đáp án, câu không có đáp án, câu lệch biên từ. Kết luận “A khác B” chỉ được đưa ra khi khoảng tin cậy không chứa 0; ngược lại báo cáo ghi “chưa đủ bằng chứng”.

3.10 Kiểm định bộ stress-test

Trước khi dùng bộ stress-test 250 câu ($5 \text{ nhóm E1-E5} \times 50$) để chấm mô hình, mỗi câu được kiểm tra tự động: (1) với câu có đáp án, đáp án vàng có nằm trong đoạn văn không; (2) `answer_start` có trở đúng vào đáp án không; (3) đáp án có xuất hiện nguyên văn trong câu hỏi không (rò rỉ đáp án); (4) số câu hỏi khác nhau và số khung câu hỏi (câu hỏi sau khi thay cụm trong nháy đơn bằng X). Một câu **hợp lệ** nếu nó không có đáp án, hoặc đáp án của nó nằm trong đoạn văn — điều kiện tối thiểu để một mô hình trích xuất *có thể* trả lời đúng. Kết quả ở Mục 4.3.

3.11 Truy vết số liệu và kiểm thử

Mọi hệ thống ghi `results/eval_<hệ thống>_<tập>.json` (số tổng, tách nhóm, commit, thời điểm, thiết bị) và `results/predictions_<hệ thống>_<tập>.json` (dự đoán từng câu). Mọi phép đo chẩn đoán đọc các tệp này. Báo cáo *không gõ tay con số kết quả nào*: script `make_report_numbers.py` sinh các macro $\LaTeX$ và các bảng từ `results/`; giá trị

nào chưa có sẽ hiện chữ đỏ “??” trong PDF thay vì một con số đoán. Mã nguồn có 218 kiểm thử tự động chạy không cần GPU, gồm các bất biến của lớp tokenizer PhoBERT, phép đo biên từ, phân loại lỗi và kiểm định McNemar.

4 Thực nghiệm và đánh giá

Nguồn của chương này. Mọi con số trong chương được sinh tự động từ các tệp `results/*.json` (Phụ lục D); không con số kết quả nào được gõ tay. Chương đi theo thứ tự: dữ liệu và công cụ đo (4.2–4.4), kết quả tổng (4.5), rồi lần lượt bốn câu hỏi nghiên cứu (4.6–4.9).

4.1 Môi trường

Bảng 7. Môi trường phần cứng và phần mềm

Thành phần	Cấu hình
Máy tính	Apple M5 Pro, 18 nhân, 48 GB bộ nhớ hợp nhất, macOS 26.4.1
Thiết bị tính toán	GPU tích hợp qua PyTorch MPS
Ngôn ngữ, môi trường	Python 3.12, uv
Học sâu	PyTorch 2.14.0 [12]; <code>transformers</code> 5.17.0 [13]; <code>tokenizers</code> 0.23.2
Tách từ	<code>pyvi</code> 0.1.1
Học máy cổ điển	<code>scikit-learn</code> 1.9.1 [11]
Kiểm thử, hình	<code>pytest</code> 9.1.1; <code>Matplotlib</code> 3.11.2

4.2 Dữ liệu

UIT-ViQuAD 2.0 được tải từ HuggingFace (`taidng/UIT-ViQuAD2.0`) bằng `scripts/fetch_data.py`. Các số trong Bảng 8 được đo trực tiếp từ tệp đã tải; độ dài tính theo âm tiết (tách theo khoảng trắng).

Bảng 8. Dữ liệu dùng trong đồ án

	train (gốc)	fit	dev	validation = test
Vai trò	—	huấn luyện	chọn epoch	chấm một lần
Số câu hỏi	28.454	27.049	1.405	3.814
Số đoạn văn	4.101	3.896	205	557
Câu không có đáp án	9.216 (32,39%)	8.773 (32,43%)	443 (31,53%)	1.161 (30,44%)
Câu có đáp án	19.238	18.276	962	2.653
Đoạn văn (âm tiết): TB / trung vị / max	179,0 / 160 / 1.537			167,6 / 152 / 618
Đáp án (âm tiết): TB / trung vị / p95	9,95 / 6 / 31			9,81 / 6 / 30

Tỉ lệ câu không có đáp án của validation là 30,44%: đó chính là điểm EM của một hệ thống *không làm gì cả* và luôn trả về chuỗi rỗng. Mọi con số dưới đây phải được đọc so với mức sàn này.

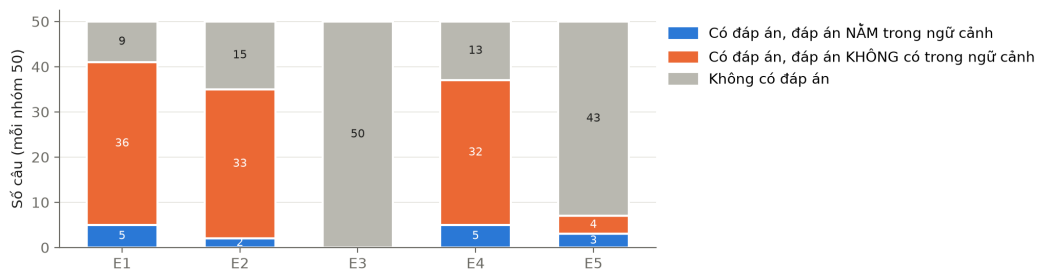
Hai tokenizer nhìn cùng dữ liệu khác nhau. Trên 400 đoạn văn train lấy ngẫu nhiên, PhoBERT (sau tách từ) sinh trung bình 171 token mỗi đoạn văn (p95: 306), XLM-R sinh 218 token (p95: 389) — chuỗi của PhoBERT ngắn hơn khoảng 22% vì một token thường là cả một từ nhiều âm tiết. Tuy vậy PhoBERT chỉ có 256 vị trí, nên 14% đoạn văn cần hơn một cửa sổ, so với 7% với XLM-R ở độ dài 384. Lợi thế “chuỗi ngắn hơn” của mô hình word-level bị giới hạn vị trí triệt tiêu một phần.

4.3 Kiểm định bộ stress-test

Bộ stress-test 250 câu được xây trong giai đoạn đầu của đề án để cô lập năm cơ chế thất bại (E1 ranh giới từ ghép, E2 ngữ cảnh gây nhiễu, E3 câu hỏi phủ định, E4 suy luận nhiều bước, E5 nhãn mập mờ). Theo giao thức ở Mục 3.10, nó được kiểm định *trước* khi chấm mô hình. Kết quả (Bảng 9, Hình 3) cho thấy bộ này **không dùng được để so sánh mô hình**.

Bảng 9. Kiểm định tự động bộ stress-test (nguồn: `results/stress_test_audit.json`)

Nhóm	Câu	đáp án	Có			Trong số câu có đáp án		Câu hỏi	
			đ.án ∈ ngữ cảnh	offset đúng	đ.án ∈ câu hỏi	khác nhau	Hợp lệ		
E1	Ranh giới từ ghép	50	41	5	2	19	21	14	
E2	Ngữ cảnh gây nhiễu	50	35	2	2	2	28	17	
E3	Câu hỏi phủ định	50	0	0	0	0	14	50	
E4	Suy luận nhiều bước	50	37	5	1	0	13	18	
E5	Nhãn mập mờ	50	7	3	1	0	8	46	



Hình 3. Thành phần từng nhóm stress-test. Phần cam là câu có đáp án mà đáp án không nằm trong ngữ cảnh — không mô hình trích xuất nào trả lời đúng được

Bốn phát hiện:

1. **Đáp án không nằm trong ngữ cảnh.** Chỉ 15 trên 120 câu có đáp án chứa đáp án vàng trong đoạn văn; chỉ 6 câu có `answer_start` đúng. Với các câu còn lại, điểm EM tối đa của *mọi* mô hình trích xuất là 0.

2. **Độ đa dạng rất thấp.** Cả 250 câu dùng chung 5 đoạn văn. Nhóm E5 có 50 câu nhưng chỉ 8 câu hỏi khác nhau; nhóm E1 có 21. Câu hỏi của E1 phần lớn là câu “siêu ngôn ngữ” dạng “*Tại sao từ ghép ‘X’ khó được tách từ chính xác trong ngữ cảnh này?*” với đáp án là chính X, nên 19 câu có đáp án nằm nguyên văn trong câu hỏi. Chúng đo khả năng chép lại cụm trong nháy đơn, không đo ranh giới từ ghép.
3. **Mất cân bằng nhãn tạo điểm thoái hoá.** E3 có 100% câu không có đáp án, E5 có 86%. Hệ thống luôn từ chối đạt 100 điểm trên E3 và 52,00 điểm trên toàn bộ 250 câu.
4. **Tập gộp chứa bản ghi ngoài lược đồ.** Tập `stress_test_combined.json` có 370 bản ghi: 250 câu SQuAD cộng 120 bản ghi “phẳng” không theo lược đồ SQuAD (24 mỗi nhóm) mà tài liệu dự án không nhắc tới. Đây là lời giải cho sự chênh lệch “370 so với 250” từng ghi nhận.

Nguyên nhân gốc. Truy ngược mã sinh dữ liệu (`legacy/stress_test_generator.py`) cho thấy hai lỗi “sai âm thầm” cụ thể: hàm `_find_answer_start` trả về 0 khi không tìm thấy đáp án trong ngữ cảnh thay vì báo lỗi; và hàm `_wrap_into_squad` gán ngữ cảnh *theo vòng* cho các mẫu chưa có ngữ cảnh — sau khi câu hỏi và đáp án đã được viết cho một ngữ cảnh khác. Hai lỗi cộng lại tạo ra dữ liệu đúng định dạng, qua được mọi kiểm tra rò rỉ (báo cáo rò rỉ đi kèm ghi `is_clean: true`), nhưng sai về nội dung.

Hệ quả cho thiết kế. Bộ stress-test được giữ lại *như một đối tượng nghiên cứu*, không như một thước đo: các mô hình vẫn được chấm trên nó (Mục 4.8) để cho thấy điểm số trên một bộ chẩn đoán lỗi trông như thế nào, nhưng không kết luận nào về mô hình dựa trên nó. Vai trò chẩn đoán được chuyển sang các phép đo trên *validation* — 2.653 câu có đáp án do người gán — bắt đầu từ phép đo biên từ dưới đây. Đây đúng là điều kiện mà Bowman và Dahl [22] nêu: một bộ chẩn đoán chỉ có giá trị khi nhãn của nó đúng.

4.4 Trần EM do biên từ

Phép đo ở Mục 3.6 chạy trên toàn bộ 2.653 câu có đáp án của validation, không cần mô hình:

- **2.620 câu (98,76%)** có ít nhất một đáp án vàng khớp biên từ của `pyvi`. Chỉ **33 câu (1,24%)** có đáp án cắt ngang một từ (13 câu cắt ở đầu, 20 câu cắt ở cuối).
- **Trần EM của một mô hình word-level hoàn hảo là 99,25%:** kênh “đơn vị đầu ra” của giả thuyết RQ1 chỉ lấy đi được tối đa 0,75 điểm EM trên câu có đáp án.
- Trên chính 33 câu lệch biên, span nguyên-từ gần nhất vẫn đạt EM trong 39,39% trường hợp, vì phần thừa chỉ là dấu câu mà bước chuẩn hoá xoá đi.

Đọc từng câu trong 33 câu lệch biên cùng kết quả tách từ xung quanh đáp án cho

một bức tranh khác hẳn giả thuyết (Bảng 10): phần lớn *không* phải là ranh giới từ ghép khó của tiếng Việt, mà là **lỗi của bộ tách từ và lỗi gán nhãn**.

Bảng 10. Phân loại 33 câu lệch biên từ trên validation (nguồn: `results/misaligned_labels.json`)

Nguyên nhân	Số câu	Ví dụ: tách từ quanh đáp án (vàng)
pyvi dính dấu chấm vào token viết hoa (luật chữ viết tắt)	13	như CNN. → CNN.; vua Ferdinand VII.
pyvi nổi nhầm âm tiết qua ranh giới từ	12	phim hoạt_hình Mỹ_Chú chuột; tại Nam_Phi_với
Lỗi gán nhãn: đáp án vàng bị cắt giữa âm tiết	5	vàng “o thời gian ...” (thiếu “d”); “...các nhà thông thá”
Ranh giới từ ghép thật	3	bởi_vì; vào_khoảng

Hệ quả cho RQ1. Kênh “PhoBERT không thể trả lời nửa từ” giải thích được *tối đa* 0,75 điểm EM trên câu có đáp án, và chỉ 3 trong 33 câu lệch biên là ranh giới từ ghép thật. Nếu PhoBERT kém XLM-R nhiều hơn mức đó, phần chênh còn lại phải đến từ kênh khác — biểu diễn, dữ liệu tiền huấn luyện, giới hạn 256 vị trí — chứ không phải từ việc đáp án cắt ngang từ ghép. Ngược lại, các lỗi của pyvi ở bảng trên (nổi nhầm âm tiết, dính dấu câu) là những lỗi *cũng xảy ra bên trong ngữ cảnh*, và chúng đi vào biểu diễn đầu vào của PhoBERT ở mọi câu, không chỉ ở câu lệch biên.

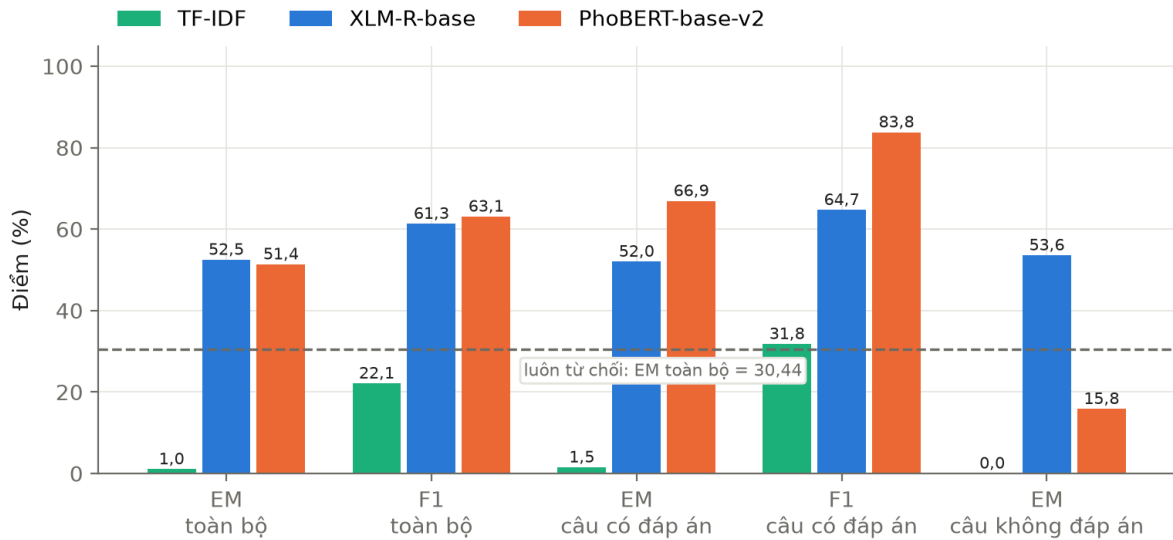
4.5 Kết quả tổng

Bảng 11. Kết quả trên toàn bộ validation (3.814 câu; nguồn: `results/eval_*_validation.json`)

Hệ thống	Toàn bộ		Có đáp án		Không đáp án	Tỉ lệ	Độ trễ
	EM	F1	EM	F1	EM	từ chối	ms/câu
Luôn từ chối	30,44	30,44	0,00	0,00	100,00	100,00	0,00
TF-IDF (truy hồi câu)	1,05	22,11	1,51	31,79	0,00	0,00	0,86
XLM-R-base	52,49	61,32	52,02	64,71	53,57	32,96	15,48
PhoBERT-base-v2	51,36	63,10	66,91	83,77	15,85	5,66	13,60

Bảng 11 cho ba quan sát.

- Hai Transformer vượt xa hai baseline.** TF-IDF có F1 22,11 nhưng EM chỉ 1,05 — thấp hơn cả hệ thống luôn từ chối (30,44) — vì nó luôn trả về cả một câu và không bao giờ từ chối.
- Điểm tổng của PhoBERT và XLM-R gần như bằng nhau:** EM 51,36 so với 52,49, F1 63,10 so với 61,32. Kiểm định cặp trên 3.814 câu cho chênh lệch EM



Hình 4. Kết quả chính. Đường đứt: điểm tổng của hệ thống luôn từ chối

-1,13 điểm, khoảng tin cậy 95% [-2,88; 0,60], p McNemar = 0,223 (Bảng 15): *không đủ bằng chứng để nói mô hình nào tốt hơn về tổng.*

3. **Nhưng điểm tổng che hai hồ sơ kỹ năng ngược nhau.** Trên câu có đáp án, PhoBERT hơn XLM-R 14,89 điểm EM ([13,08; 16,77]); trên câu không có đáp án, XLM-R hơn PhoBERT -37,73 điểm. Hai chênh lệch lớn, ngược dấu, gần như triệt tiêu nhau trong điểm tổng. Đây là minh họa trực tiếp nhất cho khoảng trống mà đề án đặt ra: một điểm tổng không cho biết mô hình sai ở đâu.

PhoBERT cũng nhanh hơn khi suy luận (13,60 so với 15,48 ms/câu trên MPS), nhất quán với chuỗi token ngắn hơn khoảng 22% (Mục 4.2), dù phải tốn thêm bước tách từ.

Kết quả hai lần chạy PhoBERT. PhoBERT được huấn luyện hai lần (Mục 4.11); lần chính (PhoBERT, lr $3 \cdot 10^{-5}$) được chọn theo F1 trên dev. Lần chạy thứ hai cho cùng hồ sơ: EM câu có đáp án 65,21, EM câu không có đáp án 13,78 (Bảng 12) — hồ sơ “đọc giỏi, hiếm khi từ chối” không phải do may rủi của một lần chạy.

Bảng 12. Hai lần chạy PhoBERT trên validation. ★: lần chạy chính (F1 dev cao nhất)

Lần chạy	Epoch	F1 dev	EM	F1	EM có đ.án	EM không đ.án	Từ chối (%)
PhoBERT, lr $3 \cdot 10^{-5}$ ★	3	60,61	51,36	63,10	66,91	15,85	5,66
PhoBERT, lr $2 \cdot 10^{-5}$	2	58,36	49,55	61,13	65,21	13,78	5,43

4.6 RQ1 — Mô hình word-level có nhạy với ranh giới từ hơn?

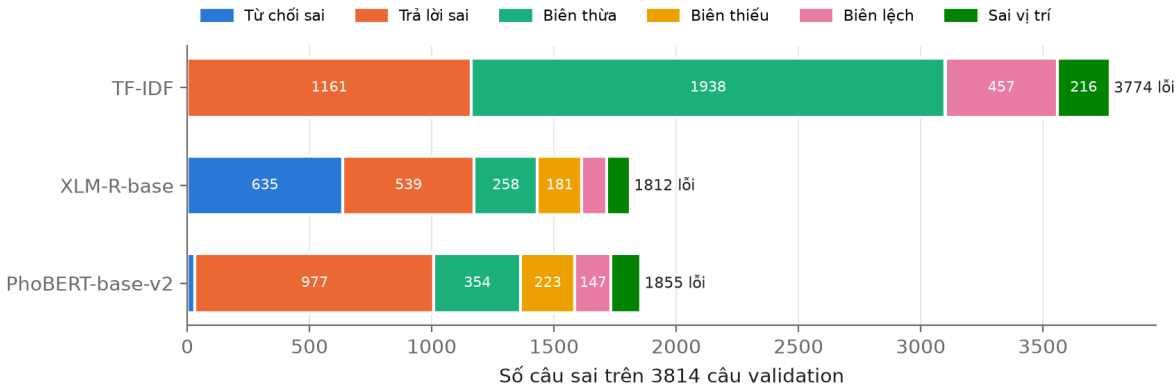
Giả thuyết dự đoán sai hướng. Giả thuyết ghi trước cho rằng PhoBERT *yếu hơn* ở ranh giới từ vì lỗi tách từ lan truyền vào mô hình. Số liệu cho điều ngược lại trên phần lớn dữ liệu: trên 2.620 câu có đáp án khớp biên từ, PhoBERT đạt EM 67,37 so với 52,10

Bảng 13. EM/F1 theo tương thích biên từ của đáp án vàng (câu có đáp án)

Hệ thống	Khớp biên (2.620 câu)		Lệch biên (33 câu)	
	EM	F1	EM	F1
Luôn từ chối	0,00	0,00	0,00	0,00
TF-IDF (truy hồi câu)	1,53	31,86	0,00	25,91
XLM-R-base	52,10	64,82	45,45	56,12
PhoBERT-base-v2	67,37	83,93	30,30	70,96

Bảng 14. Phân loại lỗi trên toàn bộ validation (số câu; định nghĩa ở Bảng 6)

Hệ thống	Tổng lỗi	Từ chối sai	Trả lời sai	Biên thừa	Biên thiếu	Biên lệch	Sai vị trí
Luôn từ chối	2.653	2.653	0	0	0	0	0
TF-IDF (truy hồi câu)	3.774	0	1.161	1.938	2	457	216
XLM-R-base	1.812	635	539	258	181	103	96
PhoBERT-base-v2	1.855	32	977	354	223	147	122



Hình 5. Cơ cấu lỗi của từng hệ thống

Bảng 15. So sánh cặp PhoBERT (A) và XLM-R (B) theo EM từng câu. Chênh lệch = $EM(A) - EM(B)$, điểm %

Nhóm câu	n	Cả hai đúng	Chỉ A	Chỉ B	Cả hai sai	Chênh lệch	CI 95%	p
Toàn bộ	3.814	1.386	573	616	1.239	-1,13	[-2,88; 0,60]	0,223
Có đáp án	2.653	1.231	544	149	729	14,89	[13,08; 16,77]	< 0,001
Không có đáp án	1.161	155	29	467	510	-37,73	[-40,83; -34,63]	< 0,001
Lệch biên từ	33	8	2	7	16	-15,15	[-33,33; 3,03]	0,180

của XLM-R (Bảng 13). Ở chính kỹ năng “xác định đúng ranh giới đáp án”, mô hình word-level tốt hơn rõ rệt.

Cơ chế giả thuyết có tồn tại — nhưng ở quy mô rất nhỏ. Trên 33 câu lệch biên từ, bức tranh đảo chiều: PhoBERT EM 30,30 so với 45,45 của XLM-R, trong khi F1 của PhoBERT lại *cao hơn* (70,96 so với 56,12). F1 cao mà EM thấp đúng là dấu vết mà cơ chế dự đoán: PhoBERT tìm đúng vùng đáp án nhưng không thể dừng ở giữa một từ. Một ví dụ điển hình: vàng “trận Gonzales”, pyvi tách thành `tại_trận Gonzales`, PhoBERT trả về “tại trận Gonzales” còn XLM-R trả về đúng. Tuy vậy, chênh lệch trên nhóm này có khoảng tin cậy $[-33,33; 3,03]$, $p = 0,180$ — chưa đủ bằng chứng thống kê với $n = 33$; và nhóm này chỉ chiếm 1,24% số câu, trong đó phần lớn là lỗi của pyvi chứ không phải ranh giới từ ghép thật (Mục 4.4).

Phân loại lỗi. Theo số đếm thô, PhoBERT mắc nhiều lỗi biên hơn (724 so với 542, Bảng 14). Nhưng số đếm thô gây hiểu nhầm: PhoBERT *trả lời* 2.621 câu có đáp án, XLM-R chỉ trả lời 2.018 câu (phần còn lại XLM-R từ chối sai). Chuẩn hoá theo số lần trả lời, tỉ lệ lỗi biên của hai mô hình **gần như bằng nhau**: 27,62% so với 26,86%. Khác biệt thật nằm ở cột từ chối: XLM-R có 635 lỗi từ chối sai (35,04% số lỗi), PhoBERT chỉ có 32; ngược lại PhoBERT có 977 lỗi trả lời câu không có đáp án (52,67% số lỗi) so với 539 của XLM-R.

Trả lời RQ1: Không — ở dạng tổng quát. Mô hình word-level không kém hơn ở ranh giới từ; nó tốt hơn 14,89 điểm EM trên câu có đáp án. Cơ chế “không trả lời được nửa từ” có thật và quan sát được, nhưng bị chặn trên ở 0,75 điểm EM và chỉ tác động lên 33 câu. Ở UIT-ViQuAD 2.0, lỗi tách từ *không* phải yếu tố quyết định khi chọn giữa hai họ mô hình; hành vi từ chối mới là yếu tố quyết định (RQ3).

4.7 RQ2 — Độ dài ngữ cảnh và độ dài đáp án

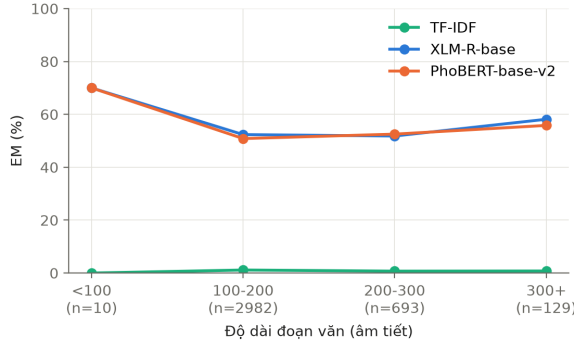
Bảng 16. EM / F1 theo độ dài đoạn văn (âm tiết). [†]: nhóm dưới 30 câu

Độ dài	n	Luôn từ chối	TF-IDF	XLM-R	PhoBERT
<100 [†]	10	0,0 / 0,0	0,0 / 31,7	70,0 / 79,9	70,0 / 84,8
100-200	2982	30,0 / 30,0	1,1 / 22,1	52,3 / 61,5	50,8 / 62,8
200-300	693	32,5 / 32,5	0,7 / 22,2	51,8 / 59,3	52,5 / 63,3
300+	129	31,8 / 31,8	0,8 / 21,3	58,1 / 66,7	55,8 / 67,1

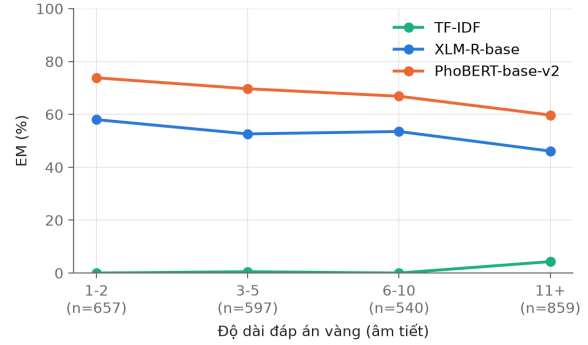
Độ dài đoạn văn không làm hiệu năng suy giảm. Với cả hai mô hình, EM không giảm khi đoạn văn dài ra (Bảng 16); nhóm trên 300 âm tiết thậm chí có điểm cao hơn, dù chỉ 129 câu và khoảng tin cậy rộng. Cửa sổ trượt tự cài làm đúng việc của nó: PhoBERT phải cắt nhiều đoạn văn thành nhiều cửa sổ hơn (14% so với 7%) nhưng không thấy dấu hiệu bị phạt. Nhóm dưới 100 âm tiết chỉ có 10 câu và không nên đọc.

Bảng 17. EM / F1 theo độ dài đáp án vàng (âm tiết; câu có đáp án)

Hệ thống	1-2 (657)	3-5 (597)	6-10 (540)	11+ (859)
Luôn từ chối	0,0 / 0,0	0,0 / 0,0	0,0 / 0,0	0,0 / 0,0
TF-IDF (truy hồi câu)	0,0 / 13,0	0,5 / 23,3	0,0 / 36,2	4,3 / 49,2
XLM-R-base	58,0 / 64,1	52,6 / 61,7	53,5 / 67,9	46,1 / 65,3
PhoBERT-base-v2	73,8 / 83,4	69,7 / 83,3	66,8 / 84,7	59,7 / 83,8



(a) Theo độ dài đoạn văn



(b) Theo độ dài đáp án

Hình 6. EM theo độ dài

Độ dài đáp án làm giảm EM, không làm giảm F1. Với cả hai mô hình, EM giảm đều khi đáp án dài ra (Bảng 17): XLM-R từ 58,0 xuống 46,1, PhoBERT từ 73,8 xuống 59,7; trong khi F1 gần như không đổi. Đáp án dài vẫn được *tìm thấy*, nhưng khớp chính xác cả hai đầu khó hơn — đó là suy giảm về độ chính xác của ranh giới, không phải về định vị. Lợi thế của PhoBERT gần như không đổi qua các nhóm độ dài, nên nó không đến từ một loại đáp án riêng nào.

Câu hỏi cần nhiều câu văn khó hơn. Theo nhãn heuristic (một câu văn phủ được $\geq 60\%$ từ nội dung của câu hỏi hay không), câu cần nhiều câu văn khó hơn rõ rệt với cả hai mô hình; khoảng cách giữa PhoBERT và XLM-R lớn hơn ở nhóm này.

Bảng 18. EM / F1 theo phạm vi suy luận (heuristic; chỉ câu có đáp án)

Loại câu hỏi	n	Luôn từ chối	TF-IDF	XLM-R	PhoBERT
single-sentence	1021	0,0 / 0,0	0,9 / 33,9	60,6 / 74,5	71,5 / 87,0
multi-sentence	1632	0,0 / 0,0	1,9 / 30,4	46,6 / 58,6	64,0 / 81,8

Trả lời RQ2: Không có suy giảm theo độ dài ngữ cảnh ở cả hai họ mô hình. Độ dài đáp án làm giảm EM (không giảm F1) theo cùng một cách ở cả hai; khác biệt giữa hai họ mô hình không phụ thuộc vào độ dài.

4.8 RQ3 — Nhận ra câu không có đáp án, hay thiên lệch về từ chối?

Bảng 19. Hành vi từ chối như một bộ phân loại “không có đáp án” (validation)

Hệ thống	Tỉ lệ từ chối (%)	Precision (%)	Recall (%)
Luôn từ chối	100,00	30,44	100,00
XLM-R-base	32,96	49,48	53,57
PhoBERT-base-v2	5,66	85,19	15,85

Tỉ lệ câu không có đáp án thật: 30,44%.

Bảng 20. EM trên bộ stress-test theo nhóm: *toàn bộ / chỉ câu hợp lệ*. Chỉ để minh hoạ — xem Mục 4.3

Hệ thống	E1	E2	E3	E4	E5	Tổng
Luôn từ chối	18,0 / 64,3	30,0 / 88,2	100,0 / 100,0	26,0 / 72,2	86,0 / 93,5	52,0
TF-IDF (truy hồi câu)	0,0 / 0,0	0,0 / 0,0	0,0 / 0,0	0,0 / 0,0	0,0 / 0,0	0,0
XLM-R-base	18,0 / 64,3	34,0 / 100,0	100,0 / 100,0	26,0 / 72,2	78,0 / 84,8	51,2
PhoBERT-base-v2	18,0 / 64,3	28,0 / 82,4	100,0 / 100,0	26,0 / 72,2	68,0 / 73,9	48,0

Số câu hợp lệ mỗi nhóm: E1 14, E2 17, E3 50, E4 18, E5 46.

Mức sàn. Hệ thống không làm gì đạt EM 30,44 trên validation. Mọi điểm trên câu không có đáp án phải được đọc so với nó.

XLM-R từ chối đúng tần suất, sai đối tượng. XLM-R từ chối 32,96% số câu — rất gần tỉ lệ thật 30,44% — nhưng precision chỉ 49,48%: gần một nửa số lần từ chối rơi vào câu có đáp án (635 câu). Có 544 câu có đáp án PhoBERT trả lời đúng mà XLM-R sai, phần lớn là những câu XLM-R từ chối — ví dụ “Ký túc xá cho sinh viên lớn nhất Paris gọi là gì?”, vàng “Cité internationale universitaire”, PhoBERT trả lời đúng còn XLM-R trả về rỗng.

PhoBERT gần như không bao giờ từ chối. PhoBERT từ chối 5,66% số câu với precision 85,19%: khi đã từ chối thì gần như luôn đúng, nhưng recall chỉ 15,85%.

Cả hai rơi vào bẫy của người gán nhãn. SQuAD 2.0 và UIT-ViQuAD 2.0 tạo câu không có đáp án bằng cách sửa câu hỏi sao cho vẫn có một cụm “nghe hợp lý” trong đoạn văn (trường `plausible_answers`). Khi PhoBERT trả lời một câu không có đáp án, 45,34% số lần (443/977) nó trả về *đúng* cụm hợp lý đó; với XLM-R là 51,39% (277/539). Hai mô hình tìm đúng thực thể khớp với câu hỏi, nhưng không nhận ra chi tiết đã bị sửa làm câu hỏi mất đáp án — tức là *không* thật sự “hiểu” rằng câu không trả lời được.

Vì sao PhoBERT mất khả năng từ chối? Loss huấn luyện toàn phần của PhoBERT *tăng* từ epoch 2 ở cả hai lần chạy (Bảng 23). Để biết phần tăng nằm ở đâu, loss của từng checkpoint được tính lại trên cùng 3.000 câu hỏi của tập fit, tách theo loại nhãn (Bảng 21). Kết quả rõ ràng: **loss trên nhãn “không có đáp án” của PhoBERT tăng khoảng 7 lần** sau epoch 1 ($0,56 \rightarrow 4,18 \rightarrow 4,28$), *trong khi* loss trên nhãn vị trí đáp án *giảm đều* ($1,35 \rightarrow 0,63$) và kết thúc thấp hơn của XLM-R. Lần chạy $lr \cdot 10^{-5}$ lặp lại y hệt. Loss “không có đáp án” của XLM-R ổn định. Tỷ lệ feature mang nhãn “không có đáp án” của hai tokenizer tương đương (39,62% so với 36,56%), nên mất cân bằng lớp không giải thích được khác biệt.

Bảng 21. Loss trên tập fit theo loại nhãn: “không có đáp án” / “vị trí đáp án” (nguồn: `results/loss_probe.json`)

Mô hình	Feature “không đ.án”	Epoch 1	Epoch 2	Epoch 3
XLM-R-base	36,6%	0,74 / 1,59	0,61 / 1,18	0,72 / 0,88
PhoBERT, $lr \cdot 10^{-5}$	39,6%	0,56 / 1,35	4,18 / 0,88	4,28 / 0,63
PhoBERT, $lr \cdot 10^{-5}$	39,6%	0,58 / 1,50	4,33 / 1,02	4,42 / 0,82

Tiêu chí chọn epoch cũng góp phần: chọn theo F1 dev thưởng cho chất lượng span, nên chọn epoch 3; chọn theo EM dev sẽ chọn epoch 1, khi PhoBERT còn giữ được khả năng từ chối. Nguyên nhân gốc vì sao PhoBERT “bỏ” lớp “không có đáp án” khi fine-tune tiếp thì đồ án *chưa xác định được*: giảm tốc độ học không khắc phục, và mất cân bằng lớp bị loại trừ; các giả thuyết còn lại (biểu diễn của token <s> trong PhoBERT, tương tác với cửa sổ 256 token, độ bất ổn số học) cần thêm thí nghiệm với nhiều seed.

Bộ stress-test không phân biệt được gì. Bảng 20 xác nhận kết luận kiểm định: ở E1, E3, E4, điểm của mọi mô hình *bằng đúng* điểm của hệ thống luôn từ chối; ở E5, hệ thống luôn từ chối *thắng* cả hai mô hình. Trên toàn bộ 250 câu, luôn từ chối (52,00) đứng trên XLM-R (51,20) và PhoBERT (48,00). Một bảng xếp hạng dựa trên bộ này sẽ kết luận rằng “không làm gì” là hệ thống tốt nhất.

Trả lời RQ3: chưa — không mô hình nào thật sự nhận ra câu không có đáp án. XLM-R từ chối đúng tần suất nhưng sai đối tượng; PhoBERT gần như không từ chối vì loss trên lớp “không có đáp án” của nó tăng vọt sau epoch 1. Cả hai trả về đúng cụm “nghe hợp lý” mà người gán nhãn đặt làm bẫy trong khoảng một nửa số lần trả lời sai.

4.9 RQ4 — Transformer có vượt baseline ở mọi nhóm?

TF-IDF thua cả hai Transformer ở *mọi* nhóm trong Bảng 16, 17, 18, và không vượt được cả mức sàn luôn từ chối về EM tổng (1,05 so với 30,44). Cơ cấu lỗi giải thích vì sao: 63,51% lỗi của nó là lỗi biên, chủ yếu “biên thừa” — nó trả về cả câu văn chứa đáp án — và nó trả lời cả 1.161 câu không có đáp án. F1 trên câu có đáp án đạt 31,79 cho thấy so khớp từ khoá thường tìm đúng *câu văn*; vai trò hợp lý của nó là bộ truy hồi đúng

trước một mô hình đọc hiểu, không phải bộ đọc hiểu.

Trả lời RQ4: Có — Transformer vượt baseline không-neural ở mọi nhóm được đo; nhưng cần một thước đo thứ hai (luôn từ chối) mới thấy rằng TF-IDF thậm chí không vượt được việc không làm gì.

4.10 Phân tích định tính

Bảng 22 là các ví dụ đại diện, lấy từ `results/disagreements_phobert_xlmr.json` (các câu hai mô hình bất đồng) và `results/error_sample_*.json` (100 lỗi ngẫu nhiên mỗi mô hình, seed 42).

Bảng 22. Ví dụ định tính trên validation

Câu hỏi	Vàng	PhoBERT	XLM-R	Hiện tượng
Ký túc xá cho sinh viên lớn nhất Paris gọi là gì?	Cité internationale universitaire	Cité internationale universitaire	∅	XLM-R từ chối sai
Người bản địa tại Nouvelle-Calédonie thuộc nhóm người nào?	Melanesia	nhóm Melanesia	∅	XLM-R từ chối sai
Người Tskhinvali chấp nhận ngưng bắn khi nào?	∅	ngày 13 tháng 8	∅	PhoBERT trả lời câu không có đáp án
Patel đạt được thành tựu lần đầu tiên vào năm nào?	∅	năm 1918	∅	bẫy “plausible” (1918)
Cuối năm 1835 trận đánh nào ở Texas đã phát triển thành xung đột vũ trang?	trận Gonzales	tại trận Gonzales	trận Gonzales	lỗi tách từ tại_trận
Các thành viên Tối cao pháp viện do ai đề cử?	cố vấn của thủ tướng và bộ trưởng tư pháp	toàn quyền bổ nhiệm theo cố vấn của ...	cố vấn của thủ tướng và bộ trưởng tư pháp	PhoBERT biên thừa

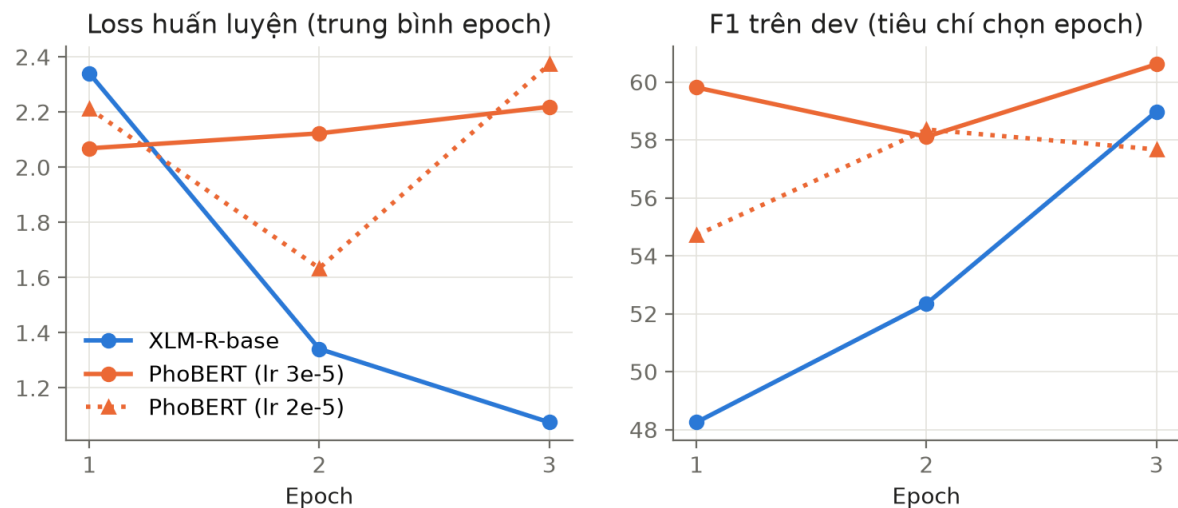
Các ví dụ khớp với các con số: lỗi đặc trưng của XLM-R là *từ chối* những câu nó lẽ ra trả lời được; lỗi đặc trưng của PhoBERT là *trả lời* những câu không có đáp án, thường bằng đúng cụm gây nhiễu; lỗi biên của PhoBERT phần lớn là thừa chữ ở đầu span, và một phần nhỏ truy được về bộ tách từ.

4.11 Quá trình huấn luyện và chi phí

XLM-R hội tụ đều: loss giảm và F1 dev tăng qua cả ba epoch; bộ chẩn đoán đường cong đánh dấu “vẫn đang cải thiện”, nên thêm epoch có thể còn tăng điểm. PhoBERT học nhanh hơn ở epoch 1 (F1 dev cao hơn XLM-R ở cùng epoch, Bảng 23), rồi loss toàn phần tăng từ epoch 2 do nguyên nhân đã phân tích ở Mục 4.8. Một epoch PhoBERT mất khoảng 22 phút, một epoch XLM-R khoảng 35 phút trên Apple M5 Pro — PhoBERT nhanh hơn vì chuỗi ngắn hơn, dù có nhiều cửa sổ hơn. Tổng thời gian huấn luyện: XLM-R 106 phút, mỗi lần chạy PhoBERT khoảng 67 phút.

Bảng 23. Đường cong huấn luyện. EM/F1 đo trên dev (500 câu ngẫu nhiên tách từ train). ★: epoch được chọn

Mô hình	Epoch	Loss	EM dev	F1 dev	Phút
XLM-R-base	1	2,3401	40,60	48,25	35,9
XLM-R-base	2	1,3411	42,40	52,34	34,8
XLM-R-base	3★	1,0751	46,40	58,98	35,3
PhoBERT, lr $3 \cdot 10^{-5}$	1	2,0684	45,20	59,81	22,8
PhoBERT, lr $3 \cdot 10^{-5}$	2	2,1226	39,40	58,11	22,2
PhoBERT, lr $3 \cdot 10^{-5}$	3★	2,2188	42,80	60,61	22,3
PhoBERT, lr $2 \cdot 10^{-5}$	1	2,2123	42,00	54,71	22,1
PhoBERT, lr $2 \cdot 10^{-5}$	2★	1,6339	39,80	58,36	23,4
PhoBERT, lr $2 \cdot 10^{-5}$	3	2,3729	38,20	57,67	22,8



Hình 7. Loss huấn luyện và EM/F1 trên dev theo epoch

5 Kết luận

5.1 Kết luận

Đề án khảo sát hai mô hình Transformer tiền huấn luyện cho đọc hiểu trích xuất tiếng Việt — PhoBERT (đơn ngữ, word-level) và XLM-R (đa ngữ, subword) — được fine-tune trong cùng điều kiện trên UIT-ViQuAD 2.0 và chấm một lần trên toàn bộ 3.814 câu validation. Bốn câu hỏi nghiên cứu được trả lời như sau.

1. **RQ1 — ranh giới từ.** Giả thuyết ghi trước (PhoBERT yếu hơn vì lỗi tách từ) *sai* ở dạng tổng quát: PhoBERT hơn XLM-R 14,89 điểm EM trên câu có đáp án, và tỉ lệ lỗi biên khi đã trả lời của hai mô hình như nhau (27,62% và 26,86%). Cơ chế “không trả lời được nửa từ” có thật, nhìn thấy được qua dấu hiệu F1 cao/EM thấp, nhưng chỉ tác động lên 1,24% câu hỏi và bị chặn trên ở 0,75 điểm EM; phần lớn các câu đó là lỗi của bộ tách từ `pyvi`, không phải ranh giới từ ghép thật.
2. **RQ2 — độ dài.** Không có suy giảm theo độ dài ngữ cảnh ở cả hai họ mô hình; độ dài đáp án làm giảm EM (không giảm F1) như nhau ở cả hai.
3. **RQ3 — câu không có đáp án.** Không mô hình nào thật sự nhận ra câu không có đáp án. XLM-R từ chối đúng tần suất nhưng sai đối tượng (precision 49,48%); PhoBERT gần như không từ chối (5,66%), vì loss trên lớp “không có đáp án” của nó tăng khoảng 7 lần sau epoch 1 trong khi loss trên vị trí đáp án vẫn giảm — lặp lại ở hai tốc độ học. Cả hai thường trả về đúng cụm gây nhiễu mà người gán nhãn đặt vào câu hỏi.
4. **RQ4 — baseline.** Transformer vượt TF-IDF ở mọi nhóm; TF-IDF thậm chí không vượt được hệ thống luôn từ chối về EM tổng.

Điều rút ra. Điểm tổng của PhoBERT và XLM-R không khác nhau có ý nghĩa thống kê, nhưng hai mô hình có hai hồ sơ kỹ năng ngược nhau: một mô hình đọc giỏi mà không biết từ chối, một mô hình biết từ chối nhưng từ chối nhầm chỗ. Chọn mô hình dựa trên điểm tổng sẽ bỏ qua điều quan trọng nhất cho triển khai. Với câu hỏi ban đầu của môn học — *đặc điểm dữ liệu quyết định phương pháp* — câu trả lời của đề án là: trên UIT-ViQuAD 2.0, đặc điểm quyết định không phải là tách từ mà là *một phần ba câu hỏi không có đáp án*; và việc một mô hình xử lý phần đó ra sao phải được đo riêng.

Về phương pháp. Hai trong ba công cụ chẩn đoán có giá trị nhất của đề án không cần chạy mô hình nào: phép đo biên từ cho biết trước giới hạn trên của hiệu ứng mà giả thuyết RQ1 dự đoán, và phép kiểm định bộ stress-test cho thấy bộ dữ liệu chẩn đoán do chính nhóm xây *không đo được thứ nó tuyên bố đo*. Nếu không kiểm định, bảng kết quả trên bộ đó sẽ xếp “không làm gì” đứng đầu mà không ai nhận ra vì sao.

5.2 Giới hạn

Các giới hạn dưới đây quyết định phạm vi mà mọi kết luận ở trên có giá trị.

1. **Một seed huấn luyện cho mỗi mô hình.** Kiểm định McNemar và bootstrap đo độ bất định do *chọn câu hỏi*, không đo độ bất định do *seed*. Với mô hình cơ base trên SQuAD, chênh lệch giữa các seed có thể lên tới một vài điểm EM; một chênh lệch nhỏ hơn cỡ đó không nên được hiểu là khác biệt giữa hai *kiến trúc*.
2. **Không tìm kiếm siêu tham số.** Cả hai mô hình dùng cùng một cấu hình phổ biến; mỗi mô hình có thể đạt điểm cao hơn ở cấu hình riêng của nó. So sánh ở đây là “cùng điều kiện”, không phải “mỗi bên ở trạng thái tốt nhất”.
3. **Bộ tách từ khác với lúc tiền huấn luyện.** PhoBERT được tiền huấn luyện trên văn bản tách bởi RDRSegmenter; đồ án dùng `pyvi` để không phụ thuộc Java. Lệnh phân phối này có thể làm PhoBERT kém hơn khả năng thật của nó, và Mục 4.4 cho thấy `pyvi` có những lỗi hệ thống (dính dấu chấm, nối nhầm âm tiết).
4. **Validation đóng vai tập test.** Test chính thức ẩn nhãn nên không chấm được. Validation không được dùng cho bất kỳ quyết định nào trước khi chấm (Mục 3.2), nhưng kết quả không so sánh trực tiếp được với các con số công bố trên test của cuộc thi VLSP.
5. **Giới hạn vị trí khác nhau là một phần của so sánh.** PhoBERT dùng 256 token, XLM-R 384 token. Đây là ràng buộc kiến trúc chứ không phải lựa chọn, nhưng nó có nghĩa là “PhoBERT vs XLM-R” không cô lập riêng biến tokenization.
6. **Bộ stress-test không dùng được làm thước đo** (Mục 4.3). Các nhóm hiện tượng E2–E5 vì vậy không có bằng chứng định lượng đáng tin cậy trong đồ án này; phân tích theo độ dài và theo loại lỗi trên validation chỉ thay thế được một phần.
7. **Phân loại thủ công 33 câu lệch biên** do một người thực hiện, chưa đo độ đồng thuận giữa các người gán. Phân loại lỗi tự động (Mục 3.7) dựa trên quan hệ tập hợp token nên tái lập được, nhưng không phân biệt được lỗi “hiểu sai câu hỏi” với “chọn nhầm thực thể cùng loại”.
8. **Miền dữ liệu.** Toàn bộ dữ liệu là Wikipedia; kết luận không tự động mở rộng sang văn bản mạng xã hội, nơi lỗi tách từ được dự đoán còn nặng hơn.

5.3 Hướng phát triển

1. **Hiệu chỉnh ngưỡng từ chối trên dev.** Ngưỡng τ trong công thức (8) được cố định bằng 0. Chọn τ riêng cho từng mô hình trên tập dev (không phải validation) là cách rẻ nhất để thử khôi phục khả năng từ chối của PhoBERT; kết quả cần được chấm lại trên validation.
2. **Tìm nguyên nhân gốc của việc PhoBERT bỏ lớp “không có đáp án”:** nhiều seed, ghi lại chuẩn gradient theo loại nhãn, thử trọng số lớp, thử độ dài 256 cho XLM-R để tách biến “độ dài cửa sổ” khỏi biến “mô hình”.

3. **Bộ tách từ.** Chạy lại PhoBERT với RDRSegmenter (VnCoreNLP) — bộ tách từ dùng lúc tiền huấn luyện — để đo phần hiệu năng bị mất do lệch bộ tách từ; và sửa luật dính dấu chấm của `pyvi`, nguyên nhân của 13 trong 33 câu lệch biên.
4. **Xây lại bộ stress-test** theo quy trình có kiểm định tự động từ đầu: đáp án phải nằm trong ngữ cảnh (báo lỗi thay vì trả về 0), mỗi nhóm có đủ câu có đáp án làm đối chứng, và câu hỏi do người viết thay vì sinh từ khuôn mẫu; đo độ đồng thuận giữa người gán.
5. **Nhiều seed** cho mọi mô hình để tách độ bất định do huấn luyện khỏi độ bất định do chọn câu hỏi.

Tài liệu tham khảo

- [1] K. V. Nguyen, D.-V. Nguyen, A. G.-T. Nguyen, and N. L.-T. Nguyen. A Vietnamese dataset for evaluating machine reading comprehension. In *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*, pages 2595–2605, 2020.
- [2] K. V. Nguyen, S. Q. Tran, L. T. Nguyen, T. V. Huynh, S. T. Luu, and N. L.-T. Nguyen. VLSP 2021 – ViMRC challenge: Vietnamese machine reading comprehension. *arXiv preprint arXiv:2203.11400*, 2022.
- [3] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2383–2392, 2016.
- [4] P. Rajpurkar, R. Jia, and P. Liang. Know what you don’t know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 784–789, 2018.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [7] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8440–8451, 2020.
- [8] D. Q. Nguyen and A. T. Nguyen. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1037–1042, 2020.
- [9] T. Kudo and J. Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of EMNLP 2018: System Demonstrations*, pages 66–71, 2018.
- [10] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [12] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [13] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations*, pages 38–45, 2020.
- [14] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [15] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

- [16] T. Vu, D. Q. Nguyen, D. Q. Nguyen, M. Dras, and M. Johnson. VnCoreNLP: A Vietnamese natural language processing toolkit. In *Proceedings of NAACL-HLT 2018: Demonstrations*, pages 56–60, 2018.
- [17] V. T. Tran. pyvi: Python Vietnamese core NLP toolkit. <https://github.com/trungtv/pyvi>.
- [18] R. Jia and P. Liang. Adversarial examples for evaluating reading comprehension systems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2021–2031, 2017.
- [19] A. Naik, A. Ravichander, N. Sadeh, C. Rose, and G. Neubig. Stress test evaluation for natural language inference. In *Proceedings of the 27th International Conference on Computational Linguistics (COLING)*, pages 2340–2353, 2018.
- [20] M. Gardner, Y. Artzi, V. Basmov, J. Berant, B. Bogin, et al. Evaluating models’ local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1307–1323, 2020.
- [21] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912, 2020.
- [22] S. R. Bowman and G. E. Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of NAACL-HLT 2021*, pages 4843–4855, 2021.
- [23] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1383–1392, 2018.
- [24] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [25] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

Phụ lục A. Hướng dẫn tái lập

Toàn bộ kết quả tái lập được bằng các lệnh sau trên một máy có Python 3.12 (GPU không bắt buộc cho phần đánh giá baseline và phân tích; bắt buộc trên thực tế cho fine-tune).

```
# 1. Môi trường và tải dữ liệu
uv venv .venv --python 3.12
uv pip install --python .venv/bin/python -r requirements.txt
python scripts/fetch_data.py                # UIT-ViQuAD 2.0 -> data/raw/

# 2. Kiểm thử (không cần GPU)
python -m pytest -m "not slow"

# 3. Phân tích không cần mô hình
python scripts/analyze_segmentation.py      # biên ửt + ấtrn EM
python scripts/audit_stress_test.py        # ểkim định stress-test

# 4. Fine-tune (dev tách ửt train; validation không được nhìn ấthy)
python scripts/finetune.py --model FacebookAI/xlm-roberta-base --out models/xlmr \
    --epochs 3 --max-length 384 --doc-stride 128 --max-answer-len 64
python scripts/finetune.py --model vinai/phobert-base-v2 --out models/phobert \
    --word-segmented --epochs 3 --max-length 256 --doc-stride 96 --max-answer-len 64

# 5. ấChm (ổmi ệh ốthng ộmt ầln, toàn ộb validation) và ấchn đoán
python scripts/run_eval.py --models abstain baseline xlmr phobert
python scripts/run_eval.py --models abstain baseline xlmr phobert --dataset stress
python scripts/diagnose.py

# 6. Báo cáo: ốs ệliu và hình sinh ửt results/, ờri biên ịdch
python scripts/make_report_numbers.py && python scripts/make_figures.py
cd report && latexmk -xelatex main.tex
```

Phụ lục B. Kiểm định khung mã ban đầu

Trước khi chạy bất kỳ thí nghiệm nào, nhóm rà soát khung mã ban đầu của đề án (nay ở *legacy/*). Rà soát cho thấy khung mã *chưa từng được chạy* (không có tệp kết quả hay checkpoint nào) nhưng tài liệu đi kèm đã trình bày số liệu. Các phát hiện được liệt kê để người đọc biết vì sao khung mã này không được dùng, và vì sao báo cáo có quy tắc “không con số nào không có tệp kết quả”.

Bảng 24. Các lỗi phát hiện trong khung mã ban đầu

Lỗi	Hệ quả nếu chạy	Xử lý
Ba bộ số liệu kết quả khác nhau (README, slide, hằng EXPECTED_RESULTS), không bộ nào có tệp nguồn	Báo cáo số không đo	Xoá toàn bộ; mọi số sinh từ results/
<code>compute_metrics</code> so sánh chuỗi " <code>span_{s}_{e}</code> " thay vì văn bản đáp án; F1 được “ước lượng” bằng $EM \times 0,95$	EM/F1 vô nghĩa	Giải mã span thật từ offset (Mục 3.4)
Giải mã PhoBERT giữ dấu “_” của bước tách từ	Đáp án đúng bị chấm EM 0	Cắt đáp án từ văn bản gốc; kiểm thử hồi quy
Ghi đè <code>max_position_embeddings</code> của PhoBERT	Hỏng embedding vị trí khi nạp	Bỏ; <code>max_length = 256</code>
<code>include_stress_test=True</code> mặc định: 60% bộ stress-test bị trộn vào tập <i>huấn luyện</i>	Rò rỉ: chấm trên dữ liệu đã học	Stress-test chỉ dùng để đánh giá
Bộ sinh stress-test: <code>_find_answer_start</code> trả về 0 khi không tìm thấy đáp án; <code>_wrap_into_squad</code> gán lại ngữ cảnh theo vòng cho mẫu không có ngữ cảnh	120–15 câu có đáp án không nằm trong ngữ cảnh (Mục 4.3)	Kiểm định; không dùng để xếp hạng

Phụ lục C. Cấu trúc mã nguồn

```

src/mrc/
  data.py, normalize.py, metrics.py      # nạp SQuAD-2.0, chuẩn hoá, EM/F1      (CS116)
  windowing.py, features.py, training.py, transformer_qa.py      (CS116)
  baseline_tfidf.py, evaluate.py, tagging.py      (CS116)
  tokenization.py      # chọn tokenizer theo loại mô hình      (CS221)
  segmented_tokenizer.py # PhoBERT: tách từ + BPE + offset về văn bản gốc      (CS221)
  segmentation.py      # thống thích biên từ, trích EM      (CS221)
  stress_test.py      # nạp + kiểm định stress-test      (CS221)
  diagnosis.py      # phân loại lỗi, từ lỗi, McNemar, bootstrap      (CS221)
  baselines.py      # luôn từ lỗi      (CS221)
scripts/
  finetune.py run_eval.py analyze_segmentation.py audit_stress_test.py
  diagnose.py make_report_numbers.py make_figures.py
tests/
  218 kiểm thử không cần GPU
results/
  eval_*.json predictions_*.json training_curve_*.json diagnosis_*.json ...
report/
  mã LaTeX tải báo cáo; generated/ do make_report_numbers.py sinh
legacy/
  khung mã ban đầu (không dùng; xem phụ lục B)

```

Phụ lục D. Bảng truy vết số liệu

Bảng 25. Nguồn gốc các nhóm số liệu trong báo cáo

Số liệu	Nguồn
Thống kê dữ liệu, kích thước train/dev	<code>data/raw/*.json</code> qua <code>compute_stats</code> , <code>split_by_context</code> (seed 42)
EM, F1, tách nhóm, độ trễ	<code>results/eval*_validation.json</code>
Điểm trên stress-test theo nhóm	<code>results/eval*_stress.json</code>
Kiểm định stress-test	<code>results/stress_test_audit.json</code>
Tương thích biên từ, trần EM	<code>results/segmentation_validation.json</code>
Phân loại lỗi, từ chối, theo độ dài đáp án, so sánh cặp	<code>results/diagnosis_validation.json</code>
Ví dụ bất đồng, mẫu lỗi định tính	<code>results/disagreements_phobert_xlmr.json</code> , <code>results/error_sample*.json</code>
Đường cong huấn luyện, siêu tham số	<code>results/training_curve*.json</code> ; nhật ký <code>results/logs/train*.log</code>
Mọi macro số và bảng sinh tự động	<code>scripts/make_report_numbers.py</code> → <code>report/generated/</code>
Hình	<code>scripts/make_figures.py</code> → <code>report/figures/</code>